Jenkins-04: Freestyle, Declarative & Scripted Pipelines with

Task 1: Freestyle Job Using Docker Containers (hiring-app)

Outcome

A Jenkins Freestyle job named hiring-app-docker-freestyle runs six sequential build steps, all executed inside Docker containers: Git Clone pulls the source, SonarQube Integration scans the code, Maven Compilation builds the artifact, Nexus Artifactory uploads the build output, Slack Notification posts the result, and Deploy on Tomcat pushes the WAR file to a running Tomcat container.

Objective

To build a Docker-based Freestyle job that needs no build tools installed on the Jenkins host itself. Every stage launches a short-lived Docker container (maven, sonar-scanner-cli, curl) so the pipeline stays reproducible and the Jenkins agent stays clean.

Prerequisites

- Jenkins master/agent with Docker Engine installed, and the Jenkins service user added to the docker group.
- Plugins installed: Git, SonarQube Scanner, Slack Notification (Nexus upload is done via mvn deploy, no dedicated plugin required).
- A running SonarQube server reachable from the Docker network, with a generated authentication token.
- A running Nexus repository manager with a hosted Maven repository and deployment credentials.
- A running Tomcat server/container with the manager-script role enabled for remote WAR deployment.
- A Slack workspace with an incoming webhook or bot token, and a target channel.
- Public read access to github.com/betawins/hiring-app.git.

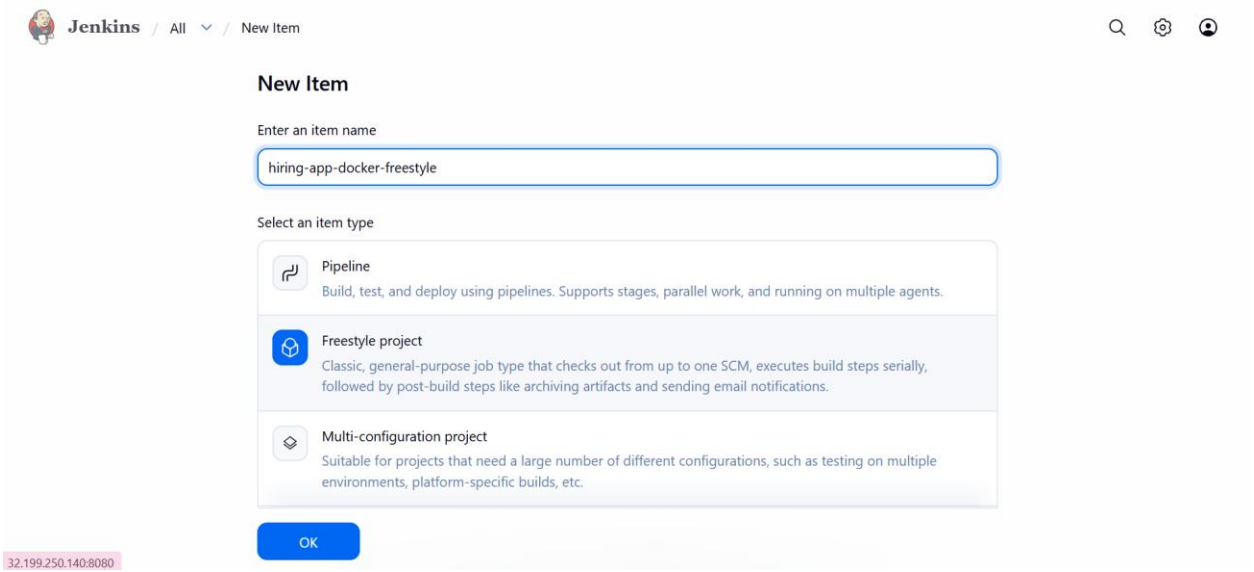
Step-by-Step Implementation

Step 1: Create the Freestyle Job and Configure Git Clone

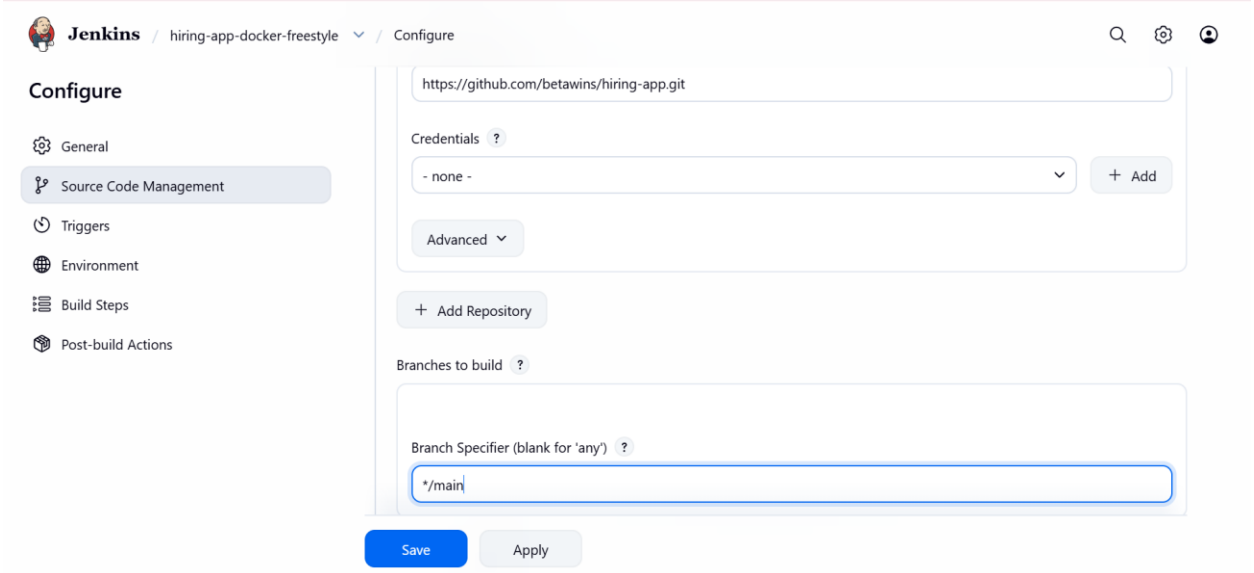
New Item -> Freestyle project -> name it hiring-app-docker-freestyle. Under Source Code Management, select Git and configure:

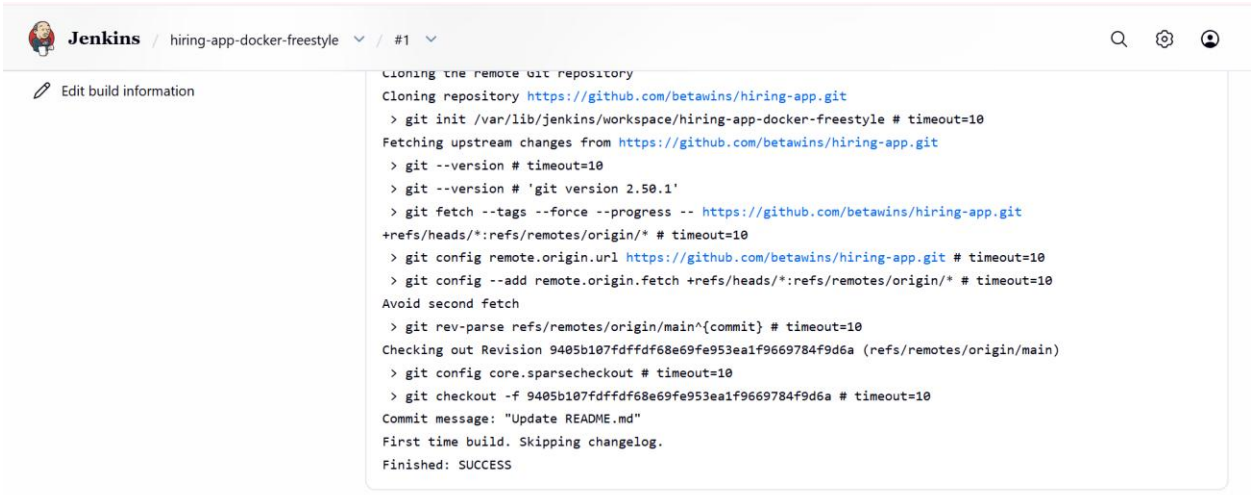
Parameter	Value
Repository URL	https://github.com/betawins/hiring-app.git
Branch Specifier	*/main

Screenshot: Git SCM config for hiring-app-docker-freestyle.



32.199.250.140:8080





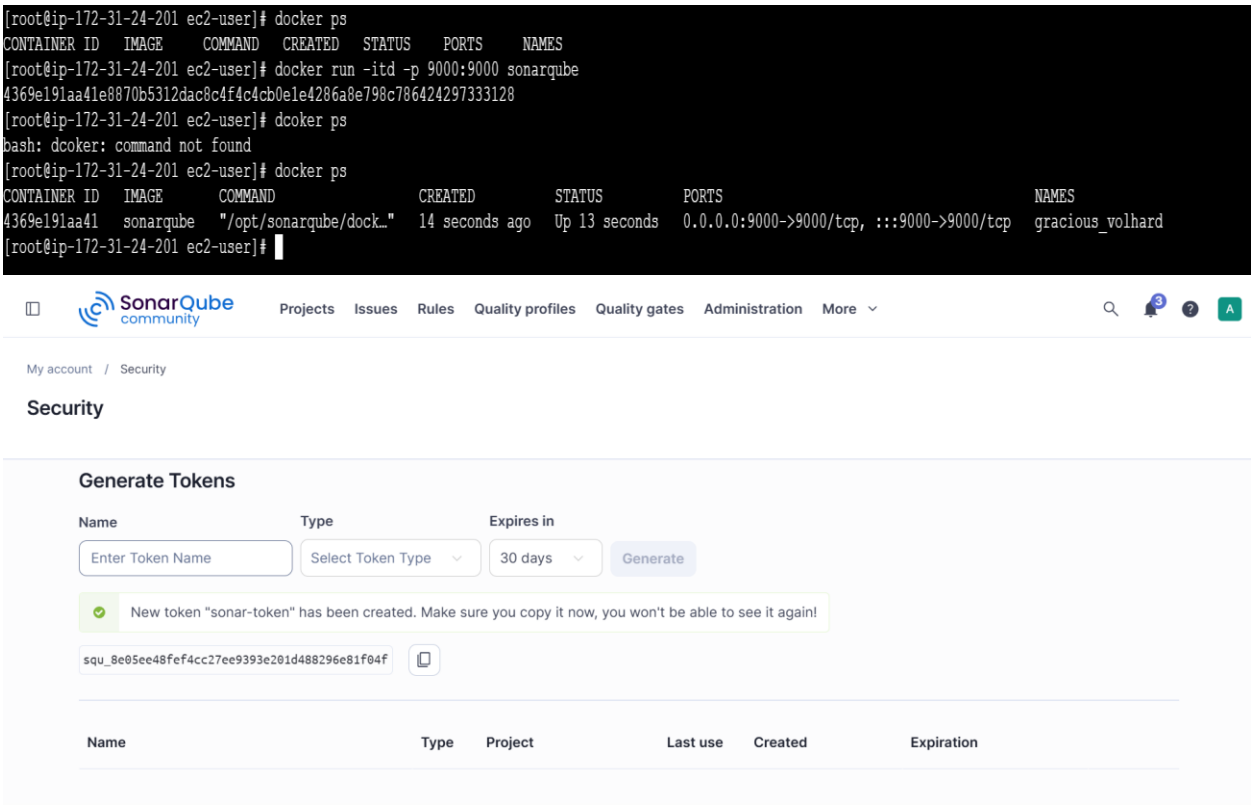
Step 2: Add a Build Step - SonarQube Integration via Docker

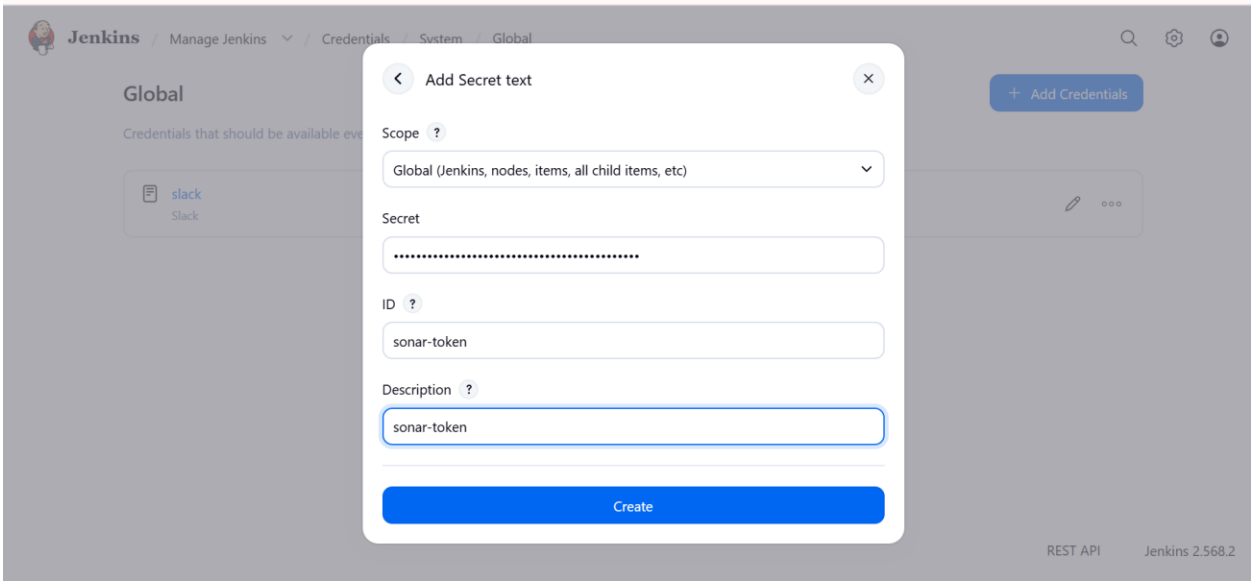
Add Build Step -> Execute shell, and run the SonarQube scanner inside a disposable container so no scanner install is needed on the host:

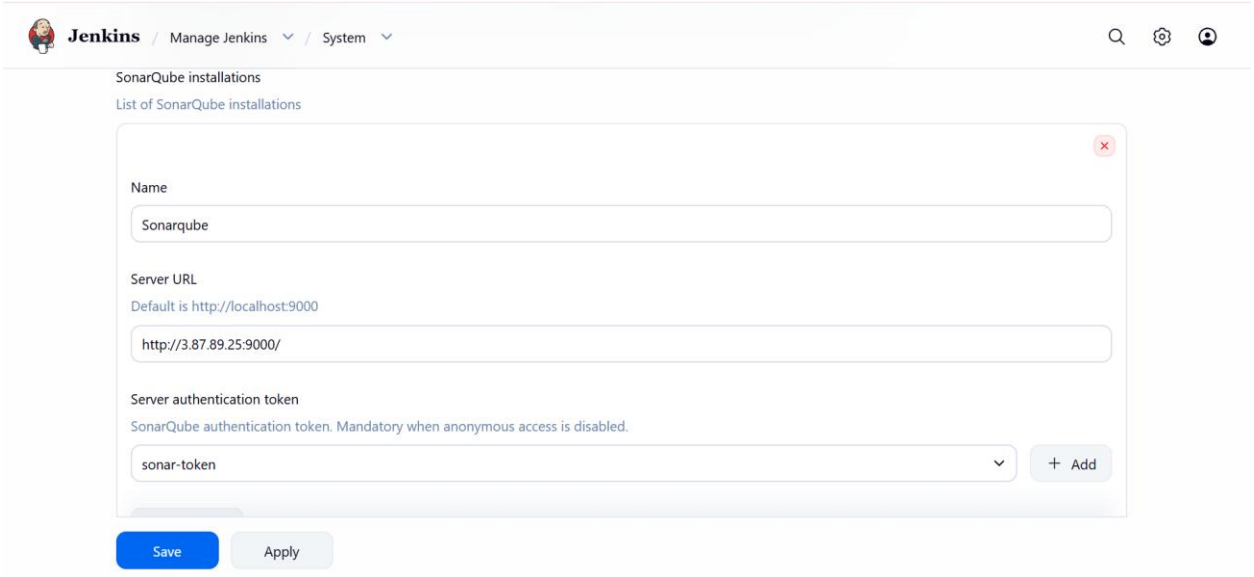
```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
docker pull sonarqube:lts-community
docker run -itd -p 9000:9000 sonarqube
```

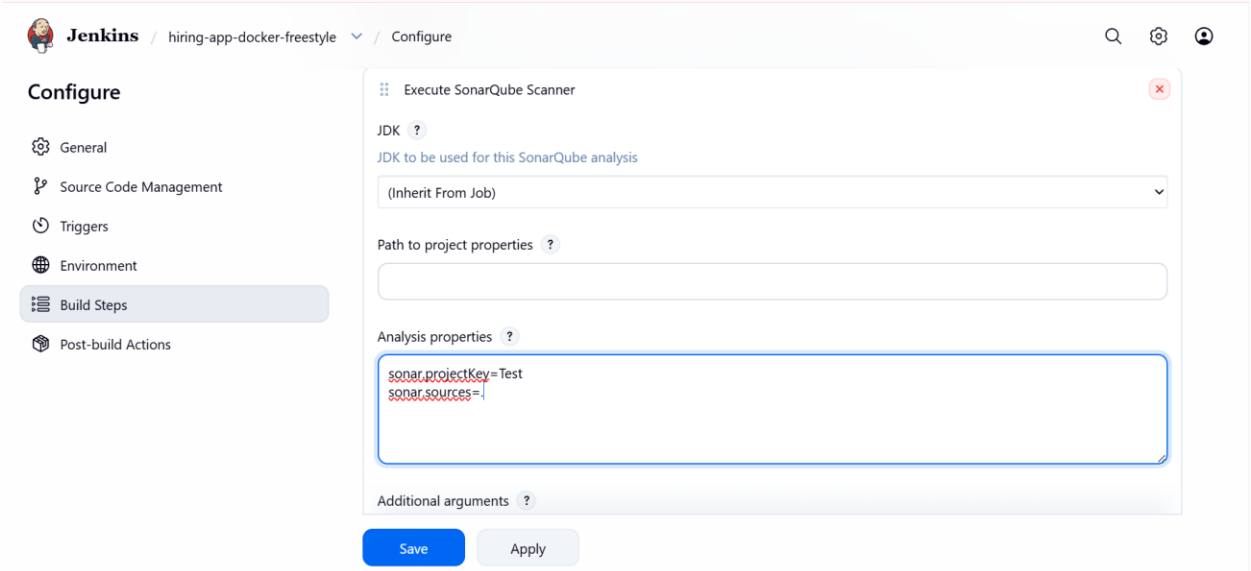
Store the SonarQube token as a Jenkins Secret text credential (ID: sonar-token).

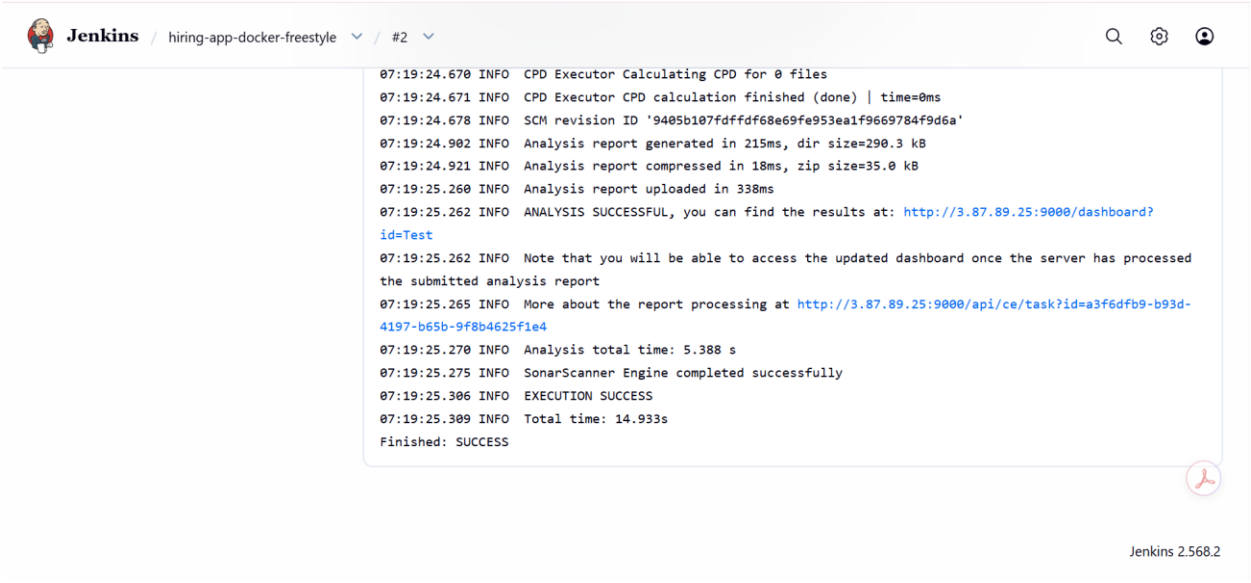
Screenshot: SonarQube Integration build step output.

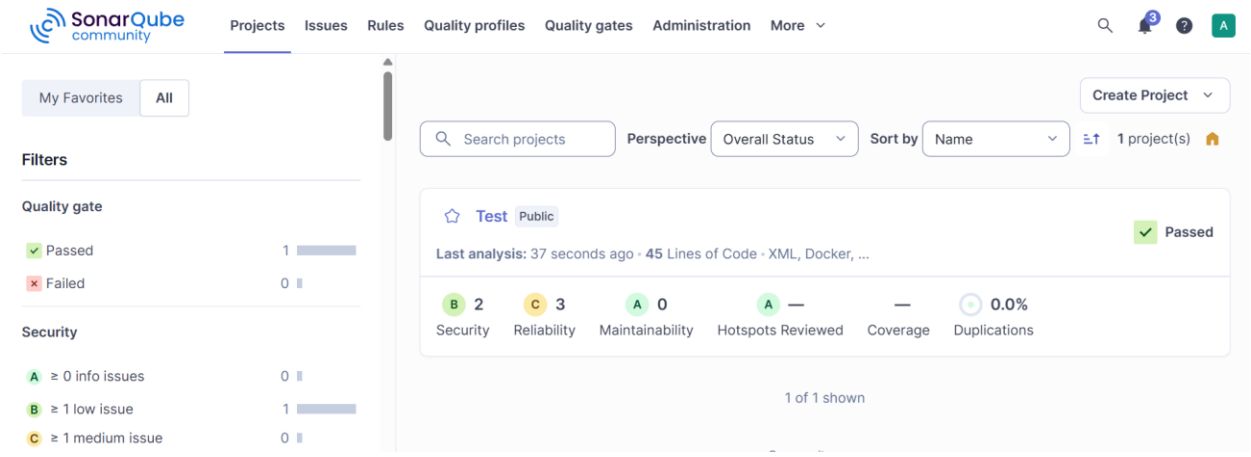



Jenkins / Manage Jenkins / Credentials / System / Global
Global
Credentials that should be available eve
slack
Slack
Add Credentials
Add Secret text
Scope
Global (Jenkins, nodes, items, all child items, etc)
Secret
ID
sonar-token
Description
sonar-token
Create
REST API
Jenkins 2.568.2


Jenkins / Manage Jenkins / System
SonarQube installations
List of SonarQube installations
Name
Sonarqube
Server URL
Default is http://localhost:9000
http://3.87.89.25:9000/
Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.
sonar-token
Add
Save
Apply





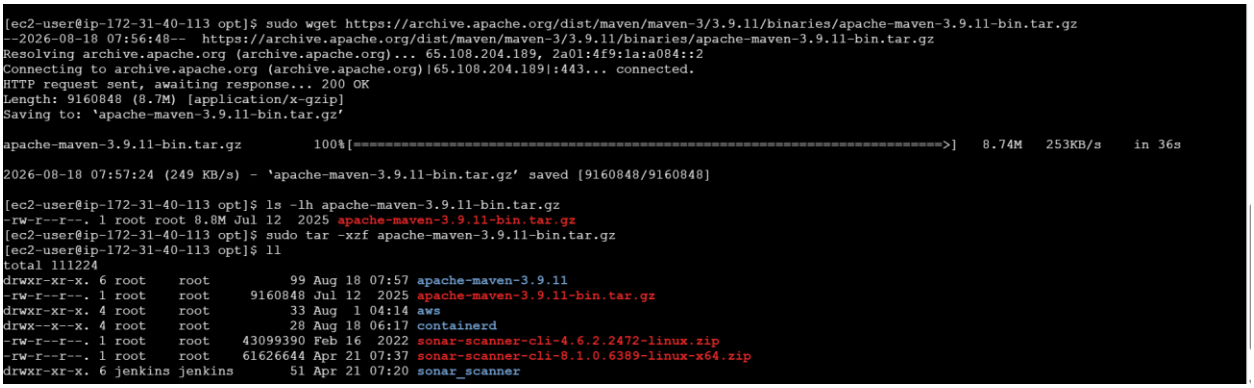


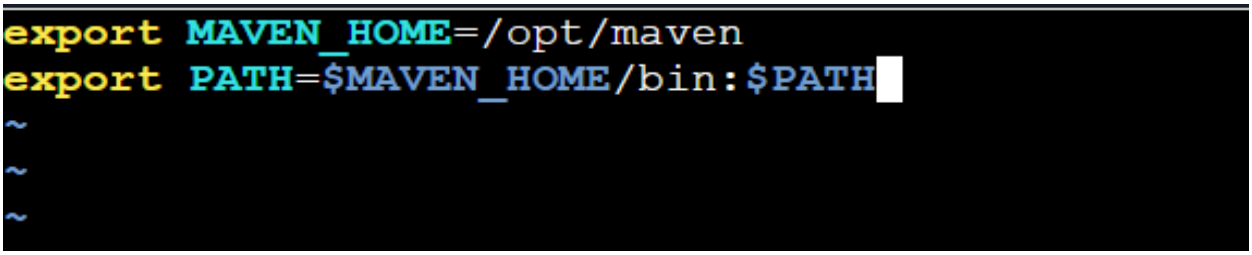
Step 3: Add a Build Step - Maven Compilation

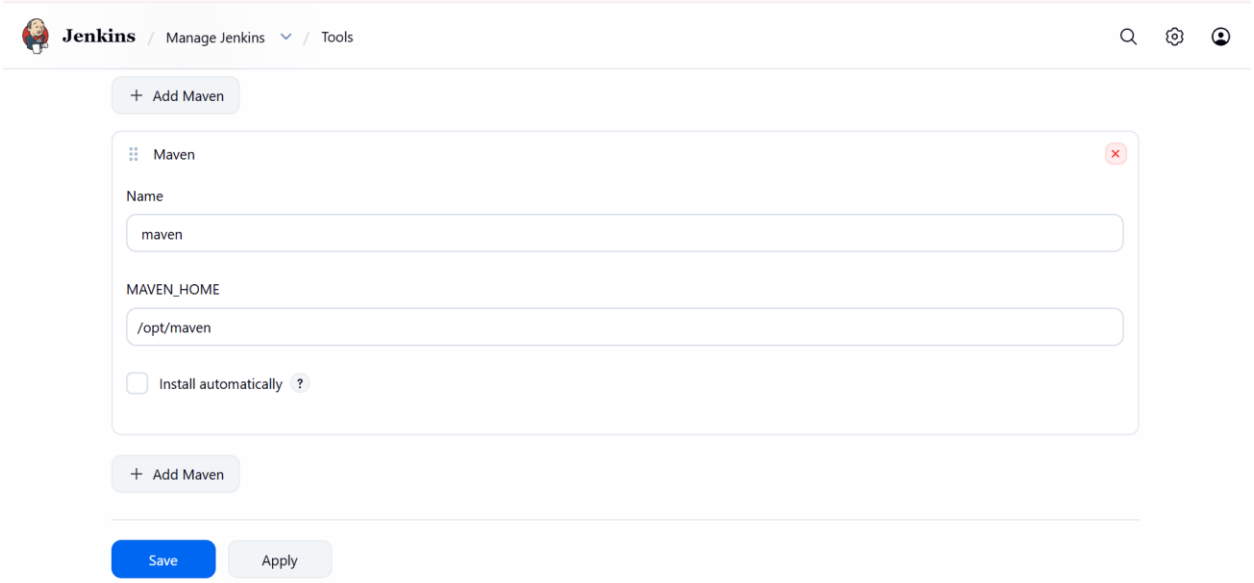
Add a second Execute shell step that compiles the project using the official host Maven install:

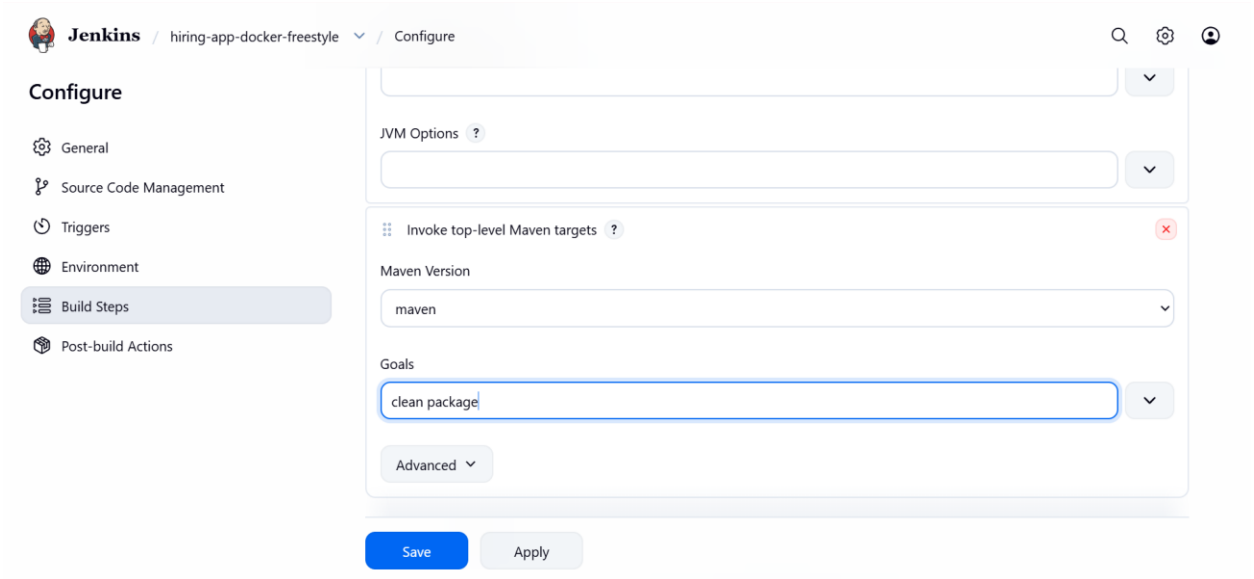
```
cd /opt
sudo wget https://archive.apache.org/dist/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.tar.gz
sudo tar -xzf apache-maven-3.9.11-bin.tar.gz
sudo mv apache-maven-3.9.11 maven
sudo vi /etc/profile.d/maven.sh
export MAVEN_HOME=/opt/maven
export PATH=$MAVEN_HOME/bin:$PATH
sudo chmod +x /etc/profile.d/maven.sh
source /etc/profile.d/maven.sh
java -version
mvn -version
```

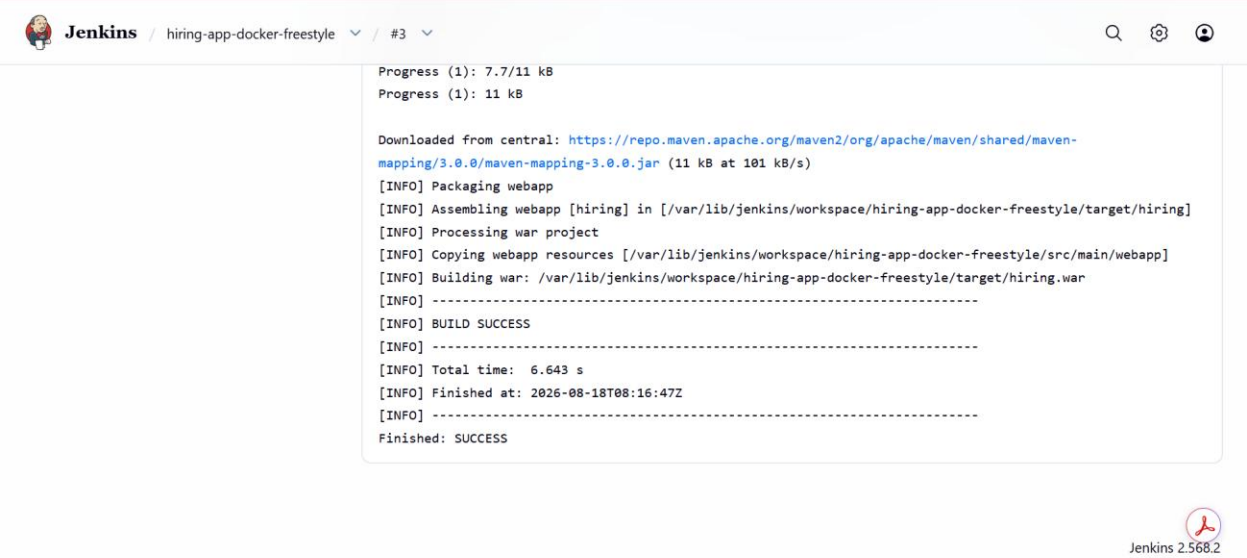
Screenshot: Maven Compilation build step output.






Jenkins / Manage Jenkins / Tools
+ Add Maven
Maven
Name
maven
MAVEN_HOME
/opt/maven
Install automatically
+ Add Maven
Save
Apply


Jenkins / hiring-app-docker-freestyle / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
JVM Options
Invoke top-level Maven targets
Maven Version
maven
Goals
clean package
Advanced
Save
Apply

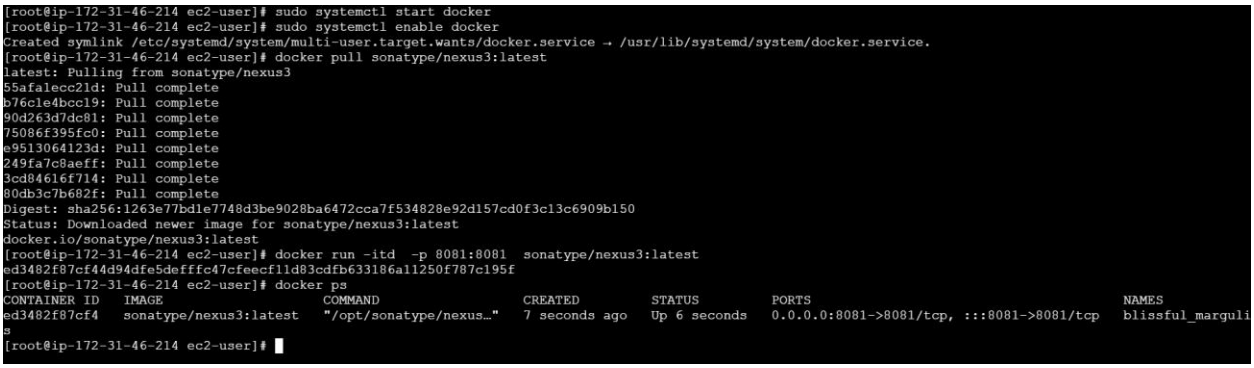


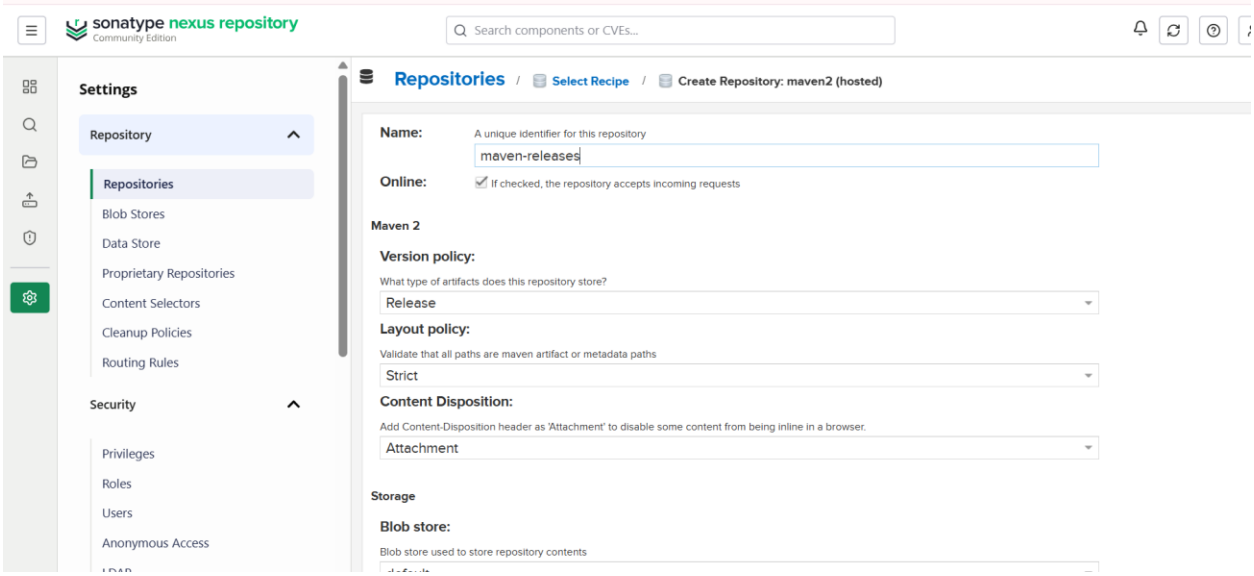
Step 4: Add a Build Step - Nexus Artifactory Upload

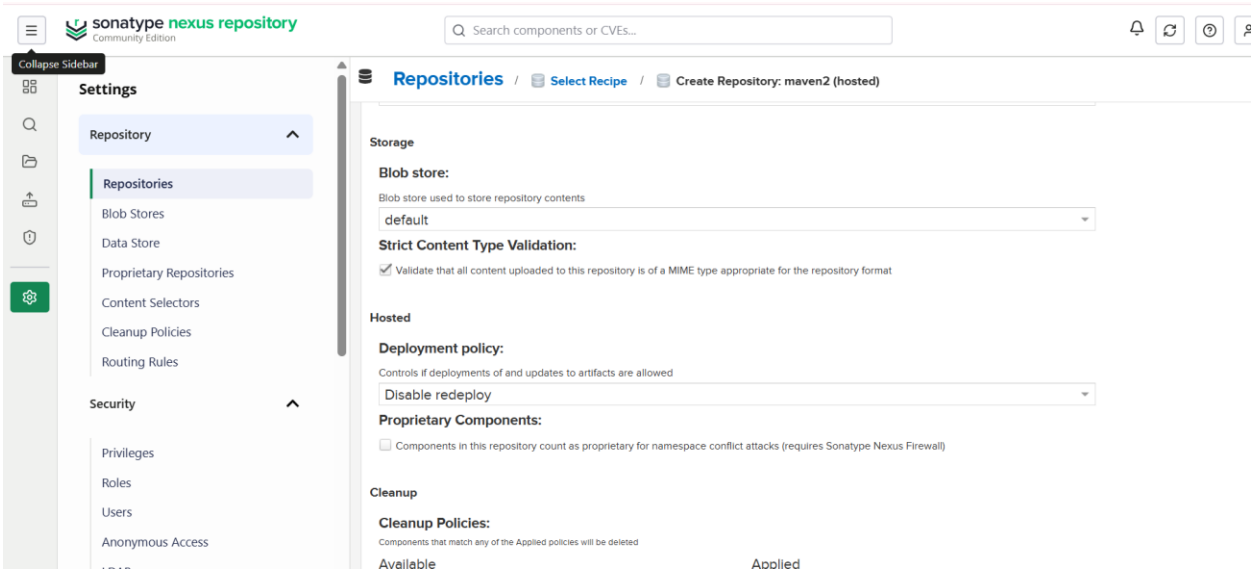
Add an Execute shell step that uploads the built artifact to Nexus, using the Maven container again so mvn deploy-file is available without a host install:

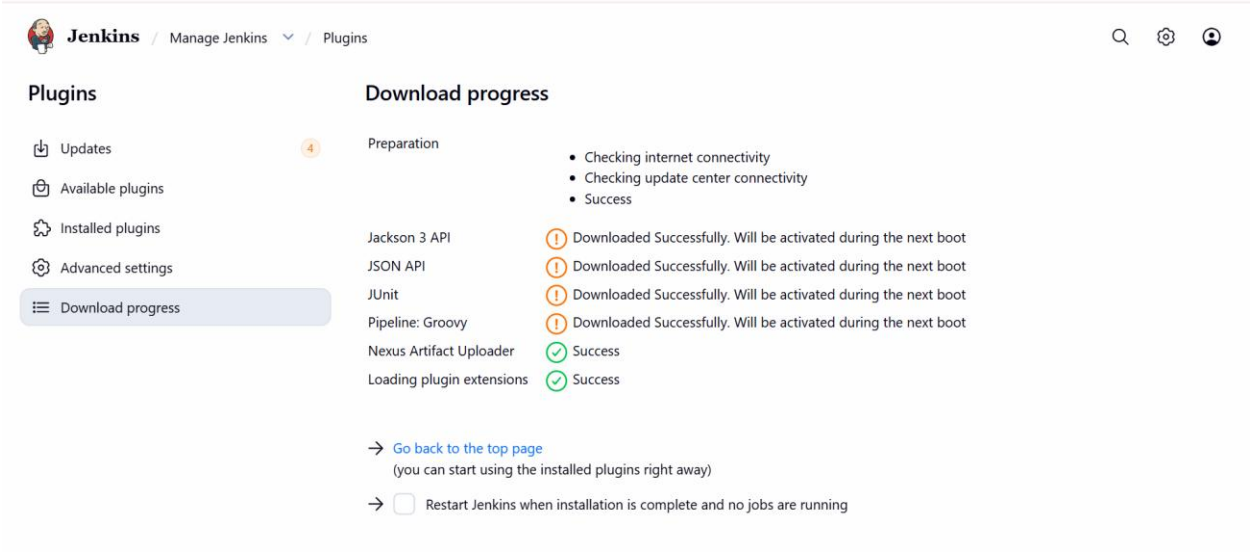
```
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
docker pull sonatype/nexus3:latest
docker run -itd  -p 8081:8081 sonatype/nexus3:latest
docker exec nexus cat /nexus-data/admin.password
```

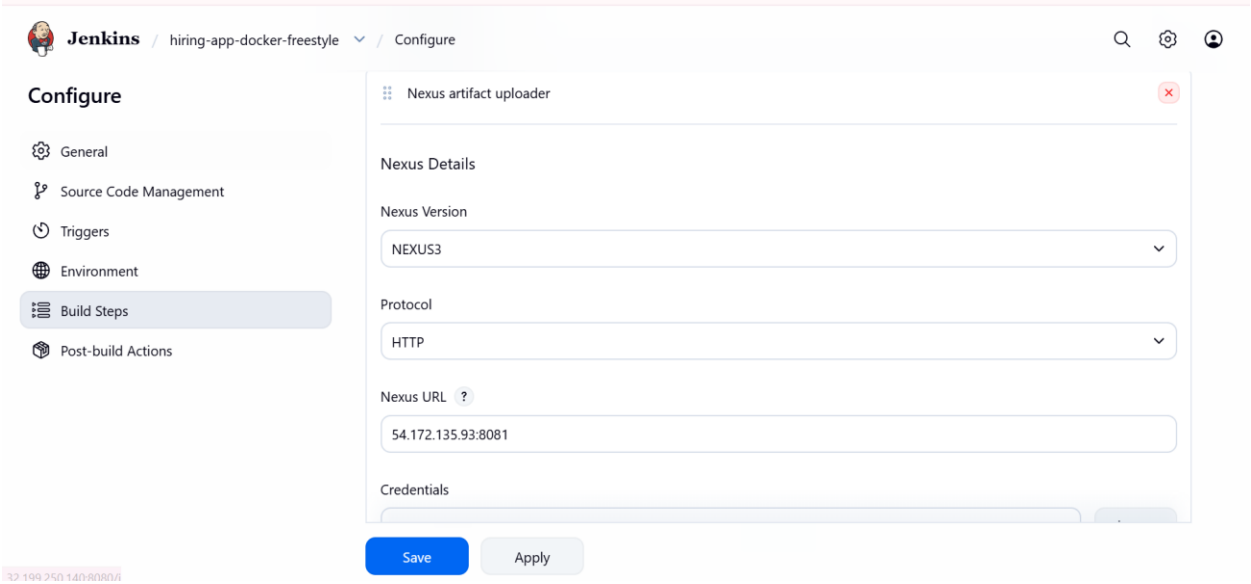
Screenshot: Nexus Artifactory upload build step output.

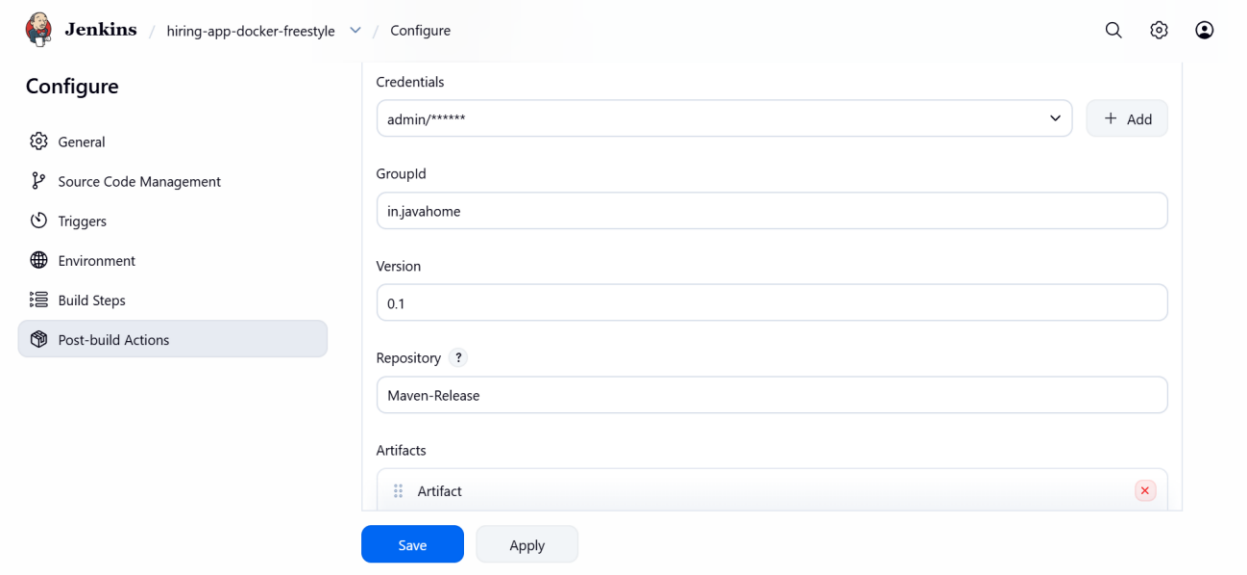


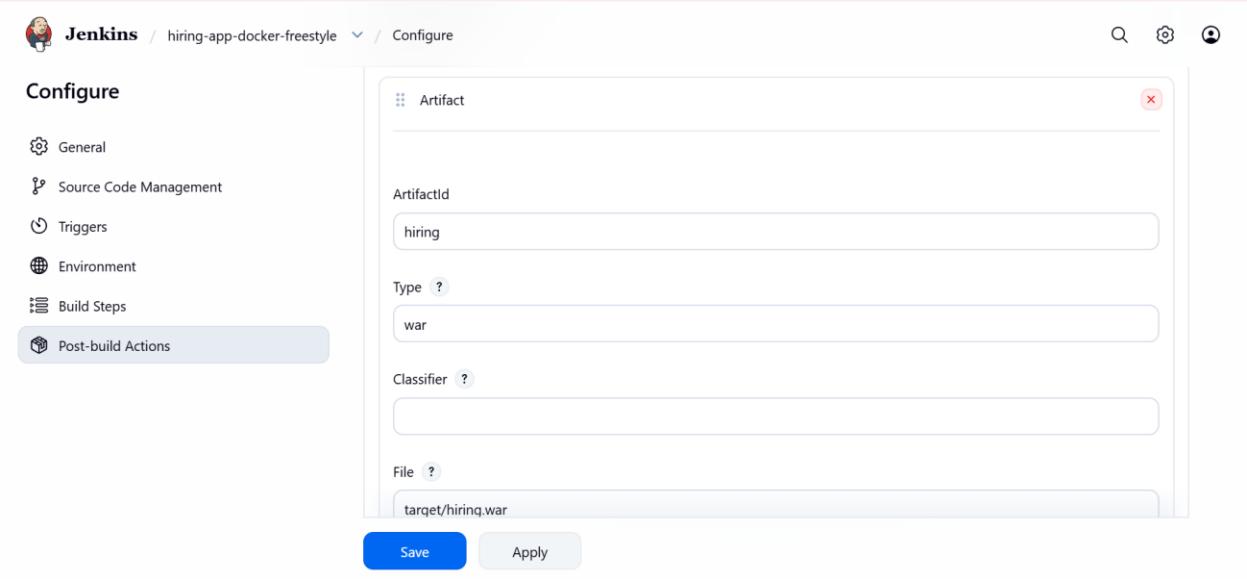

sonatype nexus repository
Community Edition
Search components or CVEs...
Settings
Repository
Repositories
Blob Stores
Data Store
Proprietary Repositories
Content Selectors
Cleanup Policies
Routing Rules
Security
Privileges
Roles
Users
Anonymous Access
Repositories / Select Recipe / Create Repository: maven2 (hosted)
Name:
A unique identifier for this repository
maven-releases
Online:
If checked, the repository accepts incoming requests
Maven 2
Version policy:
What type of artifacts does this repository store?
Release
Layout policy:
Validate that all paths are maven artifact or metadata paths
Strict
Content Disposition:
Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser.
Attachment
Storage
Blob store:
Blob store used to store repository contents

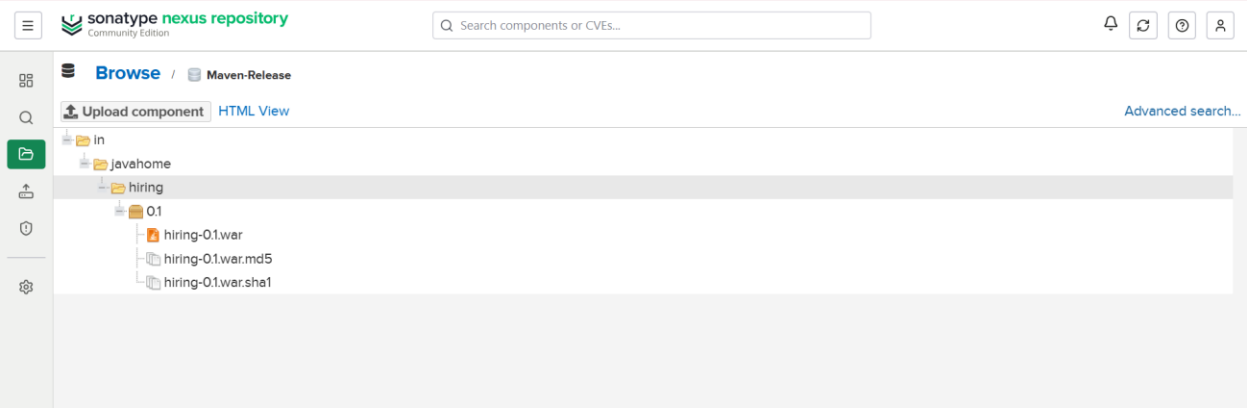

sonatype nexus repository
Community Edition
Search components or CVEs...
Collapse Sidebar
Settings
Repository
Repositories
Blob Stores
Data Store
Proprietary Repositories
Content Selectors
Cleanup Policies
Routing Rules
Security
Privileges
Roles
Users
Anonymous Access
Repositories / Select Recipe / Create Repository: maven2 (hosted)
Storage
Blob store:
Blob store used to store repository contents
default
Strict Content Type Validation:
Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format
Hosted
Deployment policy:
Controls if deployments of and updates to artifacts are allowed
Disable redeploy
Proprietary Components:
Components in this repository count as proprietary for namespace conflict attacks (requires Sonatype Nexus Firewall)
Cleanup
Cleanup Policies:
Components that match any of the Applied policies will be deleted
Available
Applied


Jenkins / Manage Jenkins / Plugins
Plugins
Updates 4
Available plugins
Installed plugins
Advanced settings
Download progress
Download progress
Preparation
Checking internet connectivity
Checking update center connectivity
Success
Jackson 3 API Downloaded Successfully. Will be activated during the next boot
JSON API Downloaded Successfully. Will be activated during the next boot
JUnit Downloaded Successfully. Will be activated during the next boot
Pipeline: Groovy Downloaded Successfully. Will be activated during the next boot
Nexus Artifact Uploader Success
Loading plugin extensions Success
Go back to the top page
(you can start using the installed plugins right away)
Restart Jenkins when installation is complete and no jobs are running


Jenkins / hiring-app-docker-freestyle / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Nexus artifact uploader
Nexus Details
Nexus Version
NEXUS3
Protocol
HTTP
Nexus URL
54.172.135.93:8081
Credentials
Save
Apply





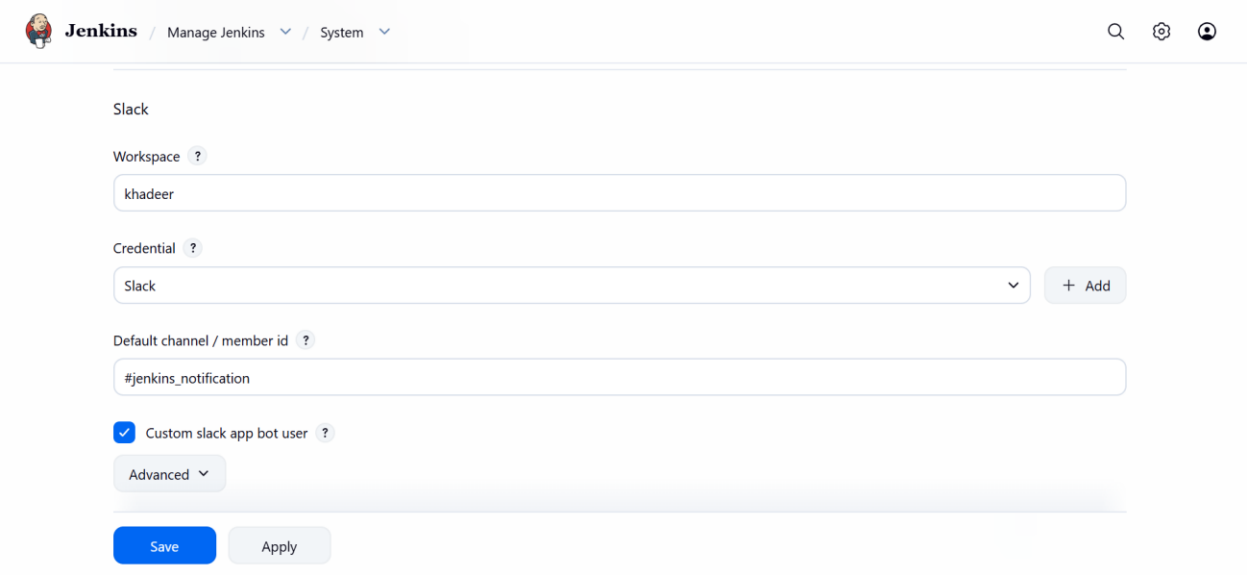


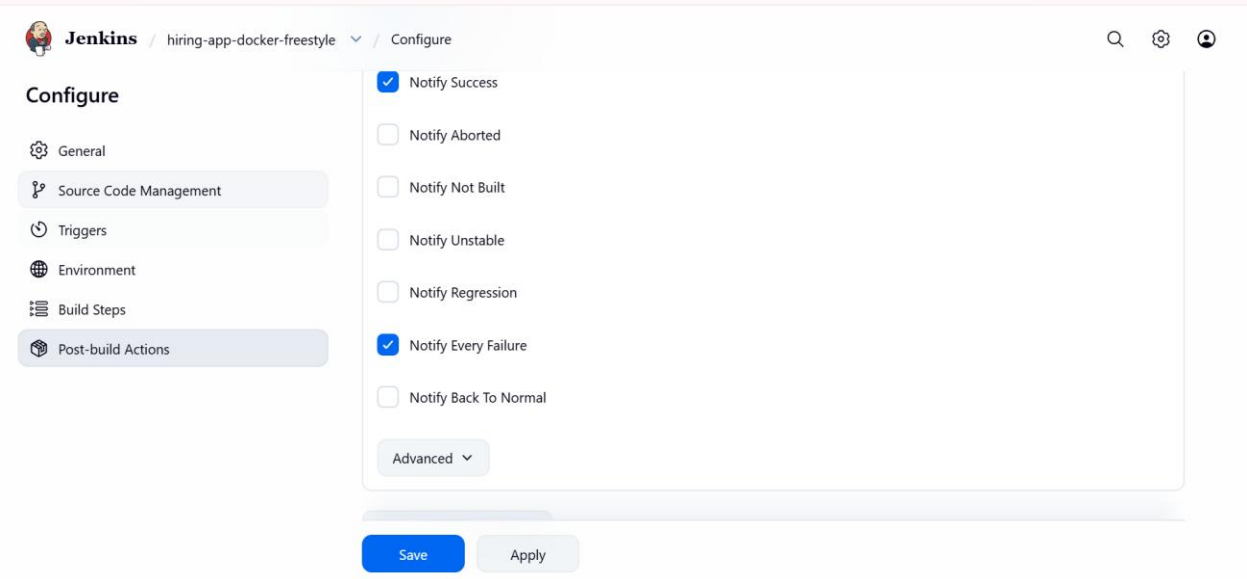
Step 5: Add a Post-Build Action - Slack Notification

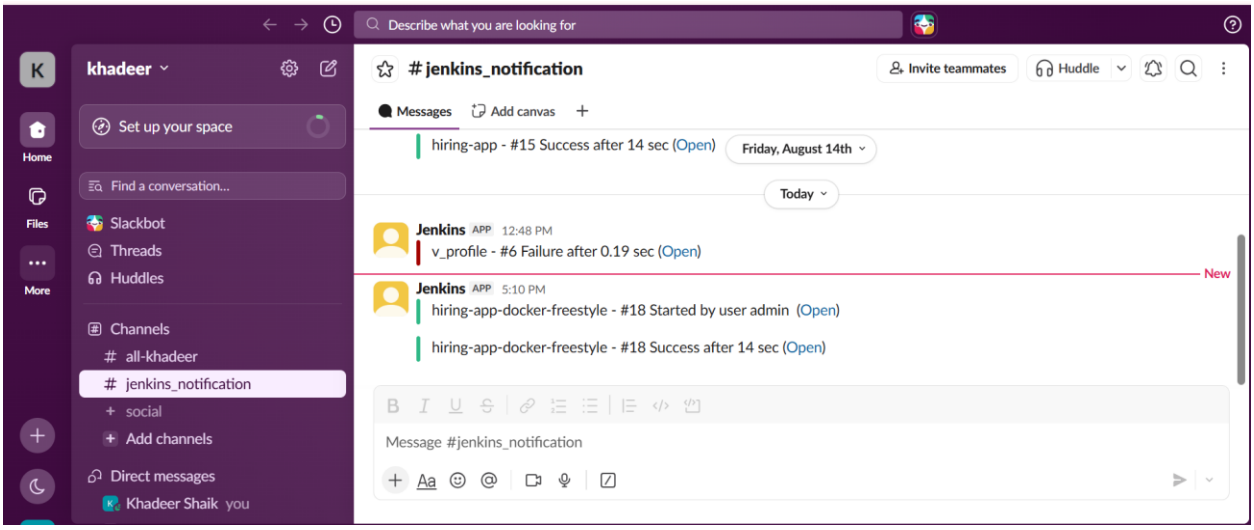
Add Post-build Actions -> Slack Notifications (from the Slack Notification plugin), or add a final Execute shell step calling curl against the Slack webhook

Parameter	Value
Location	Manage Jenkins -> System -> Slack
Credential	Slack (Secret text)
Default channel	#jenkins-notification

Screenshots : Slack Notification post-build config / Slack message.





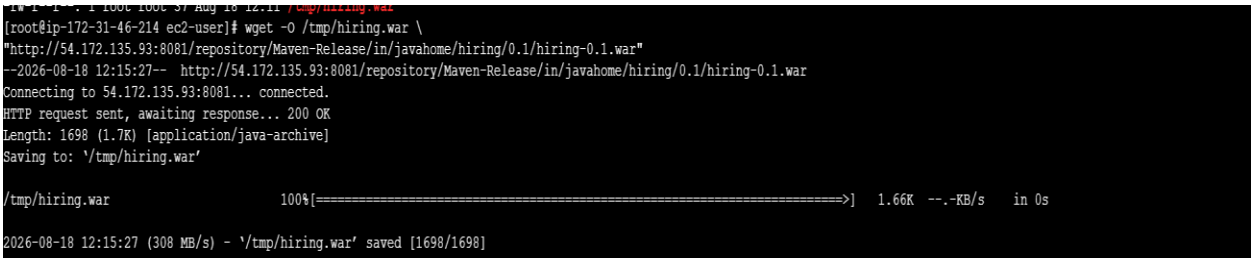


Step 6: Add a Post-Build Action - Deploy on Tomcat

Add a final Execute shell step that deploys the WAR to Tomcat's manager endpoint using a disposable curl container:

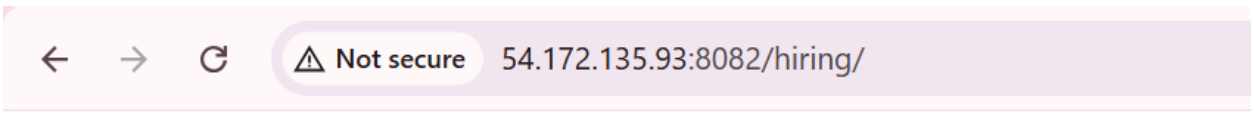
```
docker pull tomcat:10.1
docker run -d \
  --name hiring-tomcat \
  -p 8082:8080 \
  tomcat:10.1
wget -O /tmp/hiring.war \
"http://54.172.135.93:8081/repository/Maven-Release/in/javahome/hiring/0.1/hiring-0.1.war"
unzip -t /tmp/hiring.war
docker cp /tmp/hiring.war hiring-tomcat:/usr/local/tomcat/webapps/hiring.war
docker exec hiring-tomcat ls -lh /usr/local/tomcat/webapps/
```

Screenshots: Deploy on Tomcat build step output.



```
[root@ip-172-31-46-214 ec2-user]# ls -lh /tmp/hiring.war
-rw-r--r--. 1 root root 1.7K Aug 18 11:40 /tmp/hiring.war
[root@ip-172-31-46-214 ec2-user]# unzip -t /tmp/hiring.war
Archive:  /tmp/hiring.war
    testing: META-INF/MANIFEST.MF     OK
    testing: META-INF/                OK
    testing: WEB-INF/                 OK
    testing: WEB-INF/classes/         OK
    testing: WEB-INF/web.xml          OK
    testing: index.jsp                OK
    testing: META-INF/maven/in.javahome/hiring/pom.xml    OK
    testing: META-INF/maven/in.javahome/hiring/pom.properties    OK
No errors detected in compressed data of /tmp/hiring.war.
```

```
[root@ip-172-31-46-214 ec2-user]# docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS          PORTS                                                NAMES
9aa7b572edd3   tomcat:10.1-jdk21-temurin  "catalina.sh run"        29 minutes ago   Up 14 minutes   0.0.0.0:8082->8080/tcp, :::8082->8080/tcp            hiring-tomcat
8d3482f87cf4   sonatype/nexus3:latest     "/opt/sonatype/nexus…"   3 hours ago      Up 3 hours      0.0.0.0:8081->8081/tcp, :::8081->8081/tcp            blissful_margulis
[root@ip-172-31-46-214 ec2-user]# ^C
[root@ip-172-31-46-214 ec2-user]# docker cp /tmp/hiring.war hiring-tomcat:/usr/local/tomcat/webapps/hiring.war
Successfully copied 3.58kB to hiring-tomcat:/usr/local/tomcat/webapps/hiring.war
[root@ip-172-31-46-214 ec2-user]# docker exec hiring-tomcat ls -lh /usr/local/tomcat/webapps/
total 4.0K
drwxr-x---. 4 root root   54 Aug 18 12:18 hiring
-rw-r--r--. 1 root root 1.7K Aug 18 11:40 hiring.war
[root@ip-172-31-46-214 ec2-user]# docker exec hiring-tomcat ls -lh /usr/local/tomcat/webapps/hiring/
total 4.0K
-rw-r-----. 1 root root 86 Aug 18 07:06 index.jsp
drwxr-x---. 3 root root 57 Aug 18 12:18 META-INF
drwxr-x---. 3 root root 36 Aug 18 12:18 WEB-INF
[root@ip-172-31-46-214 ec2-user]#
```



Welcome to Techie Horizon

Validation Steps

- Console Output shows all six build steps completing with exit code 0, in order.
- The SonarQube dashboard shows a hiring-app project with a completed analysis and a Quality Gate result.
- The Nexus hiring-app-releases repository shows the newly uploaded WAR version.
- The configured Slack channel receives a build-result message.
- http://<tomcat-server-ip>:8080/hiring-app/ serves the deployed application.

Explanation

A Freestyle job has no native stage concept, so each pipeline stage is expressed as its own Execute shell build step, and every step launches a purpose-built, short-lived Docker container instead of relying on tools installed on the Jenkins host. This keeps the Jenkins agent lightweight and makes the toolchain versions (Maven, sonar-scanner-cli) explicit and reproducible per build.

Real-Time Use Case

Teams standardizing on containerized build tooling use this pattern to onboard legacy Freestyle jobs onto Docker without rewriting them as Pipeline jobs, keeping the Jenkins host free of per-project Maven, Sonar, and deployment tool installs while still exercising a full build-scan-store-notify-deploy flow.

Common Errors and Troubleshooting

Error	Cause	Resolution
docker: permission denied	Jenkins user not in the docker group	Add jenkins to the docker group and restart the Jenkins service
ERROR: Not authorized (SonarQube)	Missing or expired SONAR_TOKEN	Regenerate the token in SonarQube and update the bound credential
401 Unauthorized (Nexus deploy)	Wrong credentials in settings.xml	Confirm the serverId in settings.xml matches -DrepositoryId
403 Forbidden (Tomcat deploy)	manager-script role missing for the deploy user	Add manager-script to the user's roles in tomcat-users.xml

Task 2: Declarative Pipeline on the feature-1.1 Branch (sabear_simplecutomerapp)

Outcome

A Jenkins Pipeline job named simplecustomerapp-declarative, defined as a Declarative Jenkinsfile, checks out the feature-1.1 branch of sabear_simplecutomerapp and runs six stages end to end: Git Clone, SonarQube Integration, Maven Compilation, Nexus Artifactory, Slack Notification, and Deploy on Tomcat.

Objective

To express the same six-stage flow used in Task 1 using Jenkins' Declarative pipeline syntax, storing the pipeline as code (a Jenkinsfile) so it is versioned alongside the application and reviewed as part of the feature-1.1 branch.

Prerequisites

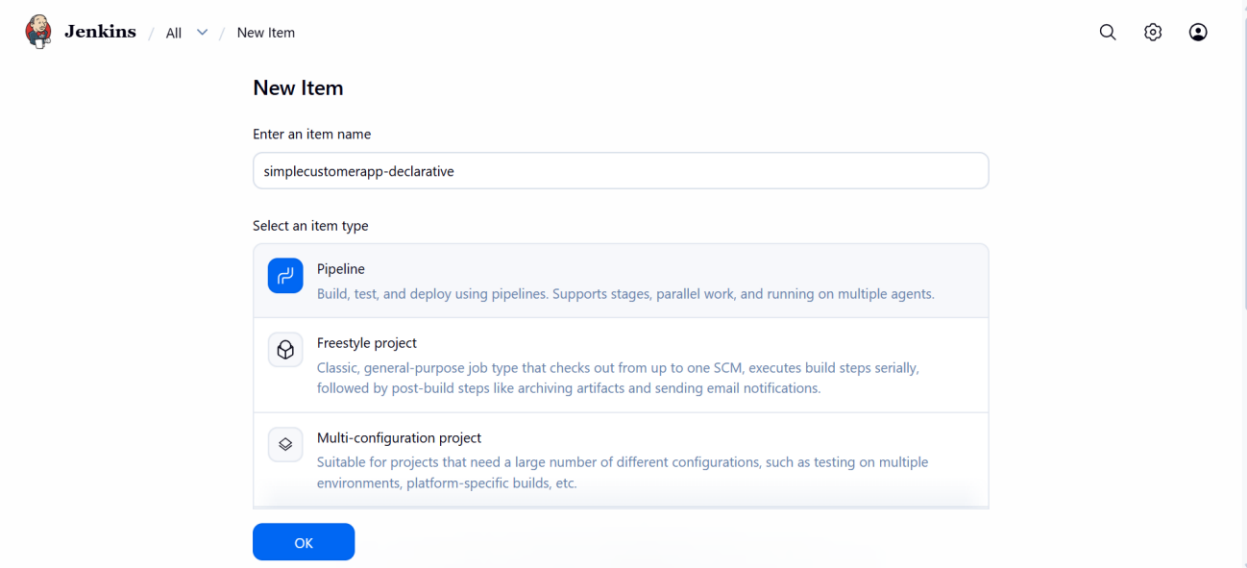
- Jenkins master with the Pipeline, Git, Pipeline: Maven Integration, SonarQube Scanner, and Slack Notification plugins installed.
- A Maven installation registered under Manage Jenkins -> Tools (name: maven-3.9), or use the tools {} directive shown below.
- SonarQube server registered under Manage Jenkins -> System -> SonarQube servers as sonar-server, with the scanner tool installed.
- Nexus repository manager reachable, with a Maven settings.xml credential stored as a Managed File (ID: nexus-settings).
- Slack integration configured under Manage Jenkins -> System -> Slack, credential slack-token.
- Tomcat manager credentials stored as a Jenkins Username/Password credential (ID: tomcat-creds).
- Read access to github.com/betawins/sabear_simplecutomerapp.git and the feature-1.1 branch.

Step-by-Step Implementation

Step 1: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	simplecustomerapp-declarative
Definition	Pipeline script from SCM (or Pipeline script pasted directly)

Screenshots: Pipeline job creation (simplecustomerapp-declarative).



Step 2: Add the Declarative Jenkinsfile

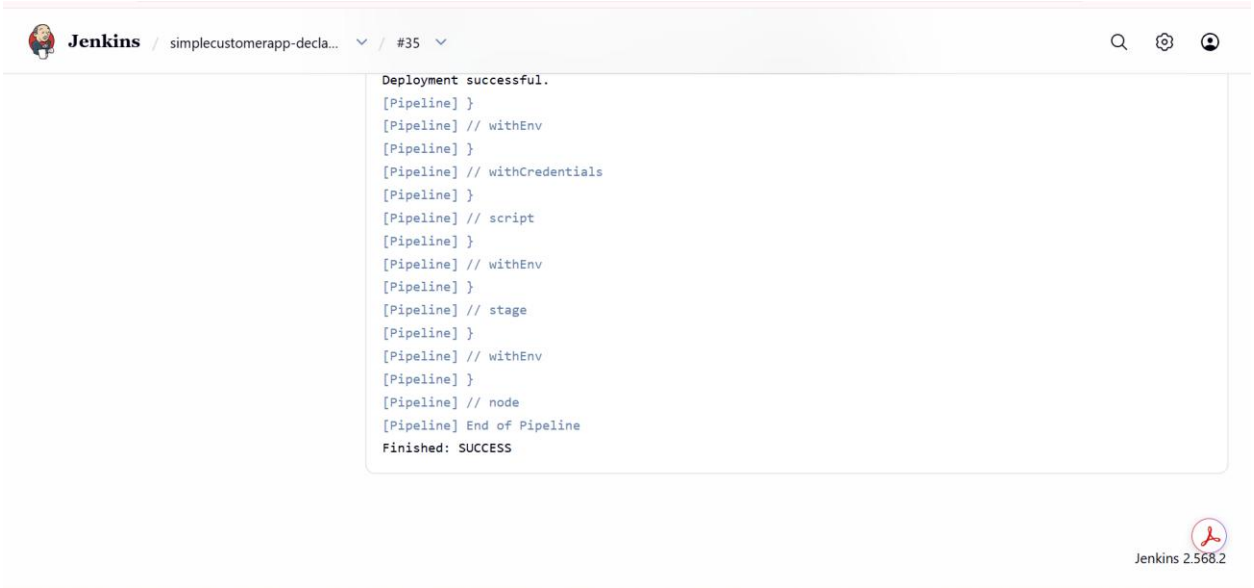
Add a Jenkinsfile at the repository root on the feature-1.1 branch, or paste the script directly into the job, with all six stages:

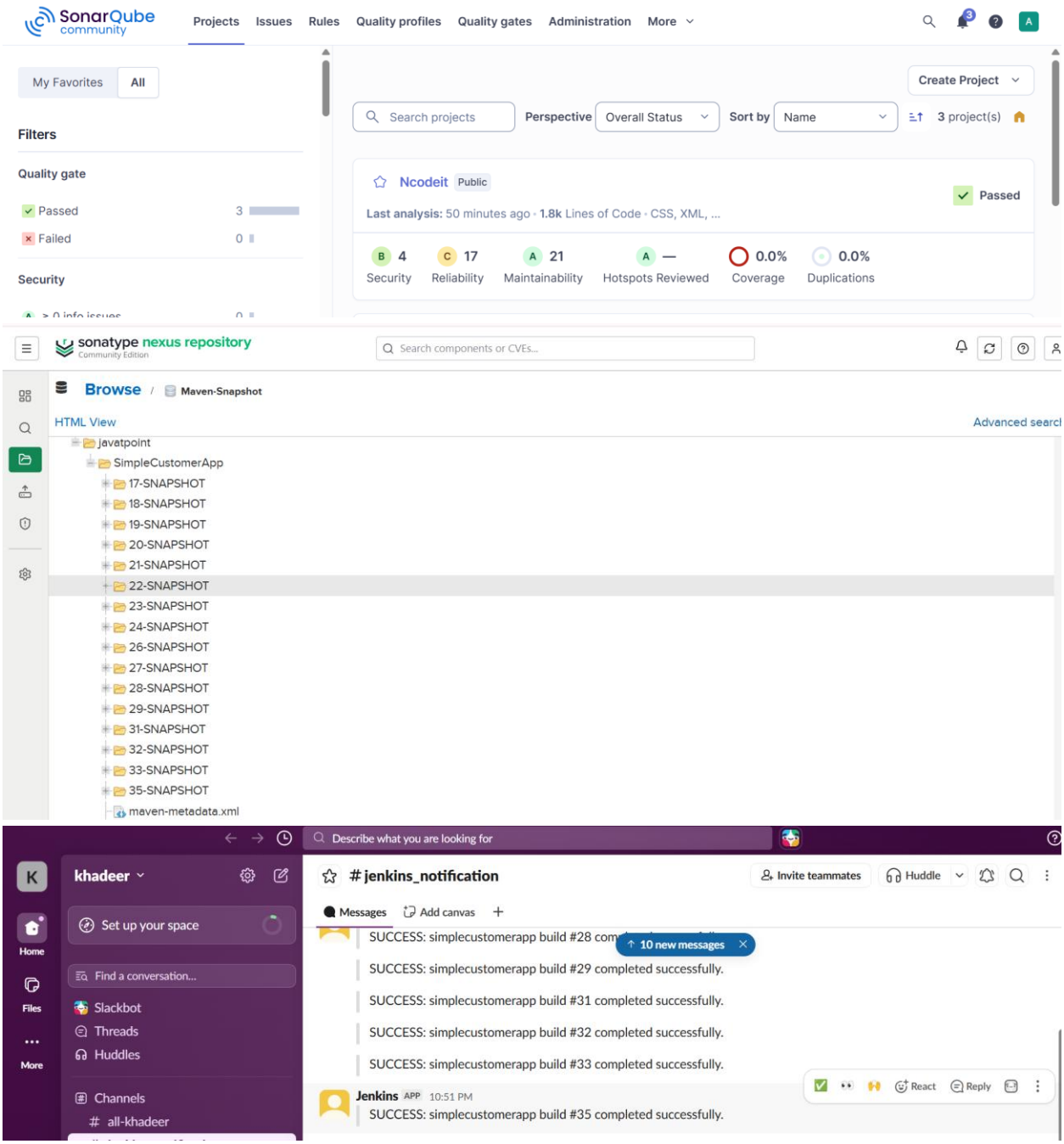
```
pipeline {

    agent any

    tools {
        maven 'maven'
    }

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'feature-1.1',
                    url: 'https://github.com/betawins/sabear_simplecutomerapp.git'
            }
        }

stage('SonarQube Integration') {
            steps {
                withSonarQubeEnv('Sonarqube') {
                    sh '''
```

```
                    /opt/sonar_scanner/bin/sonar-scanner \
                    -Dsonar.projectKey=Ncodeit \
                    -Dsonar.projectName=Ncodeit \
                    -Dsonar.projectVersion=2.0 \
                    -Dsonar.sources=.
                '''
            }
        }
    }

stage('Maven Compilation') {
        steps {
            sh '''
                mvn versions:set -DnewVersion=${BUILD_NUMBER}-SNAPSHOT
                mvn clean package
            '''
        }
    }
stage('Nexus Artifactory') {
  steps {
    withCredentials([
      usernamePassword(
        credentialsId: 'nexus',
        usernameVariable: 'NEXUS_USER',
        passwordVariable: 'NEXUS_PASSWORD'
      )
    ]) {
      sh '''
        cat > settings.xml <<EOF
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>${NEXUS_USER}</username>
      <password>${NEXUS_PASSWORD}</password>
    </server>
```

```
    </servers>
</settings>
EOF

                mvn deploy \
                    -DskipTests \
                    -s settings.xml \
                    -DaltDeploymentRepository=nexus::http://34.229.108.174:8081/repository/maven-snapshots/

                rm -f settings.xml
            '''
        }
    }
    }
    }
}
```

Screenshots: Declarative Jenkinsfile added to the job.

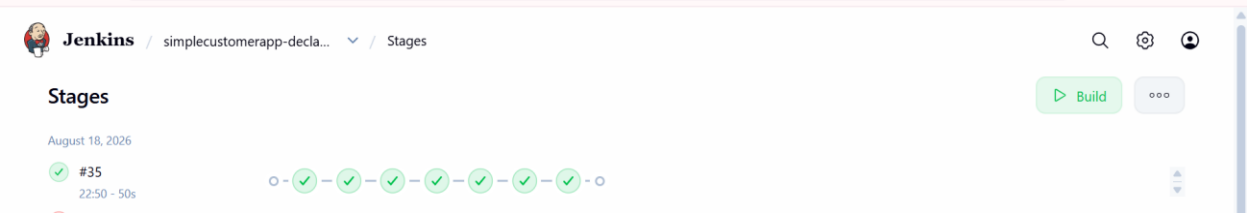




Step 3: Run the Job and Review Results

Click Build Now and open Console Output to watch all six stages execute in order under the Stage View, then confirm SonarQube, Nexus, Slack, and Tomcat each reflect the new build.

Screenshots: Console Output / Stage View of a completed run.



Validation Steps

- Stage View shows Git Clone, SonarQube Integration, Maven Compilation, Nexus Artifactory, Slack Notification, and Deploy On Tomcat all completed in green.
- The checked-out workspace matches the feature-1.1 branch (verify with git rev-parse --abbrev-ref HEAD in Console Output).
- SonarQube shows a completed analysis for the simplecustomerapp project.
- Nexus shows the new WAR version under simplecustomerapp-releases.
- Slack receives the build-result message, and http://<tomcat-server-ip>:8080/simplecustomerapp/ serves the app.

Explanation

Declarative syntax wraps every stage inside a single pipeline {} block with a fixed top-level structure (agent, tools, environment, stages, post), which Jenkins can validate and render in the Stage View without executing it first. Branch selection is explicit in the Git Clone stage (branch: 'feature-1.1'), and shared values like the Nexus URL and Slack channel are centralized in the environment {} block instead of being repeated in every stage.

Real-Time Use Case

Feature branches commonly carry their own Jenkinsfile so a change on feature-1.1 is validated, scanned, packaged, and deployed to a feature-specific environment before merge, giving reviewers a working deployment and a SonarQube report to check alongside the code review.

Common Errors and Troubleshooting

Error	Cause	Resolution
Branch not found	Branch specifier does not match feature-1.1 exactly, or shallow clone excludes it	Confirm branch: 'feature-1.1' and that the branch exists on the remote
mvn: command not found	tools {} block name does not match the Maven install name	Match the tools { maven '...' } name to Manage Jenkins -> Tools exactly
No such DSL method 'slackSend'	Slack Notification plugin not installed or job not restarted after install	Install/enable the plugin, then restart Jenkins or the agent
Deploy step returns 404	Context path in the Tomcat URL does not match the WAR name	Match ?path=/simplecustomerapp to the deployed WAR's context

Task 3: Scripted Pipeline on the feature-1.1 Branch (sabear_simplecutomerapp)

Outcome

A Jenkins Pipeline job named simplecustomerapp-scripted, defined as a Scripted Jenkinsfile, checks out the feature-1.1 branch of sabear_simplecutomerapp and runs the same six stages as Task 2 - Git Clone,

SonarQube Integration, Maven Compilation, Nexus Artifactory, Slack Notification, Deploy on Tomcat - using Groovy's node/stage syntax with explicit try/catch error handling.

Objective

To implement the identical six-stage flow using Scripted pipeline syntax, showing the same outcome achieved with imperative Groovy control flow instead of the fixed Declarative structure, including a try/finally block so Slack is notified even if an earlier stage fails.

Prerequisites

- Same plugin set as Task 2: Pipeline, Git, SonarQube Scanner, Slack Notification, plus Pipeline: Groovy.
- Same external services as Task 2: SonarQube server (sonar-server), Nexus repository, Tomcat manager, and Slack integration, all already configured and reusable.
- Read access to github.com/betawins/sabear_simplecutomerapp.git and the feature-1.1 branch.

Step-by-Step Implementation

Step 1: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	simplecustomerapp-scripted
Definition	Pipeline script (Groovy, Scripted)

Step 2: Add the Scripted Jenkinsfile

Paste the following Scripted pipeline into the job's Pipeline script field, or commit it as Jenkinsfile.scripted on the feature-1.1 branch:

```
node {

    // =========================================================
    // VARIABLES
    // =========================================================
   def APP_NAME = "simplecustomerapp"       // Sonar/display name
def ARTIFACT_NAME = "SimpleCustomerApp"  // Maven WAR name
def SONAR_PROJECT_KEY = "simplecustomerapp"
def BRANCH = "feature-1.1"

def SONAR_HOST = "http://50.17.59.76:9000"
def SONAR_SCANNER = "/opt/sonar_scanner/bin/sonar-scanner"

def NEXUS_URL = "34.229.108.174:8081"
def NEXUS_REPOSITORY = "simpleCustomerApp"

def TOMCAT_HOST = "34.229.108.174"
def TOMCAT_DEPLOY_PATH = "/opt/tomcat/webapps"


    try {

        // =====================================================
        // 1. GIT CLONE
        // =====================================================
```

```
        stage('Git Clone') {

            echo "Cloning branch: ${BRANCH}"

            git branch: BRANCH,
                url: 'https://github.com/betawins/sabear_simplecutomerapp.git'

            echo "Git clone completed successfully"
        }


        // ====================================================
        // 2. SONARQUBE INTEGRATION
        // ====================================================

stage('SonarQube Integration') {
    echo "Starting SonarQube analysis"

    withCredentials([
        string(
            credentialsId: 'sonar-token',
            variable: 'SONAR_TOKEN'
        )
    ]) {
        sh """
            ${SONAR_SCANNER} \
            -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
            -Dsonar.projectName=${APP_NAME} \
            -Dsonar.sources=src \
            -Dsonar.host.url=${SONAR_HOST} \
            -Dsonar.token=\${SONAR_TOKEN}
        """
    }

    echo "SonarQube analysis completed"
}


        // ====================================================
        // 3. MAVEN COMPILATION
        // ====================================================

stage('Maven Compilation') {
        def mvnHome = tool 'Maven'

        echo "Using Maven from: ${mvnHome}"

       sh "${mvnHome}/bin/mvn clean package -DskipTests"
    }


  // ================================================
        // 4. GET MAVEN VERSION
        // ================================================
stage('Get Version') {
```

```
    def mvnHome = tool 'Maven'

    env.APP_VERSION = sh(
        script: """
            ${mvnHome}/bin/mvn \
            help:evaluate \
            -Dexpression=project.version \
            -q \
            -DforceStdout
        """,
        returnStdout: true
    ).trim()

    echo "Application Version: ${env.APP_VERSION}"
}


        // ================================================
        // 5. NEXUS ARTIFACTORY
        // ================================================

stage('Nexus Artifactory') {
    echo "Uploading artifact to Nexus"

    def warFile = "target/SimpleCustomerApp-${env.APP_VERSION}.war"

    sh """
        echo "Application version: ${env.APP_VERSION}"
        echo "Checking WAR file..."
        ls -lh target/

        test -f "${warFile}"
    """

    nexusArtifactUploader(
        nexusVersion: 'nexus3',
        protocol: 'http',
        nexusUrl: NEXUS_URL,
        groupId: 'com.javatpoint',
        version: env.APP_VERSION,
        repository: NEXUS_REPOSITORY,
        credentialsId: 'nexus-credentials',
        artifacts: [[
            artifactId: 'SimpleCustomerApp',
            classifier: '',
            file: warFile,
            type: 'war'
        ]]
    )

    echo "Artifact uploaded to Nexus successfully"
}



        // ================================================
```

```
    // 5. SLACK NOTIFICATION
    // ====================================================

    stage('Slack Notification') {

        slackSend(
            channel: 'C0BQWR8R0QG',
            color: 'good',
            message: """
                BUILD SUCCESS

                Application : ${APP_NAME}
                Branch      : ${BRANCH}
                Version     : ${env.APP_VERSION}
                Build       : #${env.BUILD_NUMBER}

                Nexus upload completed successfully.
            """
        )
    }


    // ====================================================
    // 6. DEPLOY ON TOMCAT
    // ====================================================

stage('Deploy On Tomcat') {

echo "Deploying application to Tomcat"

sh '''
    echo "Jenkins workspace:"
    pwd

    echo "Available WAR files:"
    find target -maxdepth 1 -type f -name "*.war" -print
'''

def warFile = sh(
    script: 'find target -maxdepth 1 -type f -name "*.war" | head -1',
    returnStdout: true
).trim()

if (!warFile) {
    error "No WAR file found in Jenkins workspace"
}

echo "Deploying WAR: ${warFile}"

withCredentials([
    sshUserPrivateKey(
        credentialsId: 'nexus-tomcat-ssh',
        keyFileVariable: 'SSH_KEY',
        usernameVariable: 'SSH_USER'
    )
]) {
```

```
      sh """
        scp -i "\${SSH_KEY}" \
          -o StrictHostKeyChecking=no \
          "${warFile}" \
          \${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

        ssh -i "\${SSH_KEY}" \
          -o StrictHostKeyChecking=no \
          \${SSH_USER}@${TOMCAT_HOST} '
            sudo rm -rf ${TOMCAT_DEPLOY_PATH}/${APP_NAME}
            sudo rm -f ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo cp /tmp/${APP_NAME}.war \
              ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo chown tomcat:tomcat \
              ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo systemctl restart tomcat
          '
      """
    }

    echo "Application deployed successfully on Tomcat"
}


    } catch (err) {

      // =====================================================
      // FAILURE NOTIFICATION
      // =====================================================

      echo "Pipeline failed: ${err}"

      slackSend(
        channel: 'C0BQWR8R0QG',
        color: 'danger',
        message: """
          BUILD FAILED

          Application : ${APP_NAME}
          Branch      : ${BRANCH}
          Build       : #${env.BUILD_NUMBER}

          Check Jenkins console output for details.
        """
      )

      throw err
    }
}
```

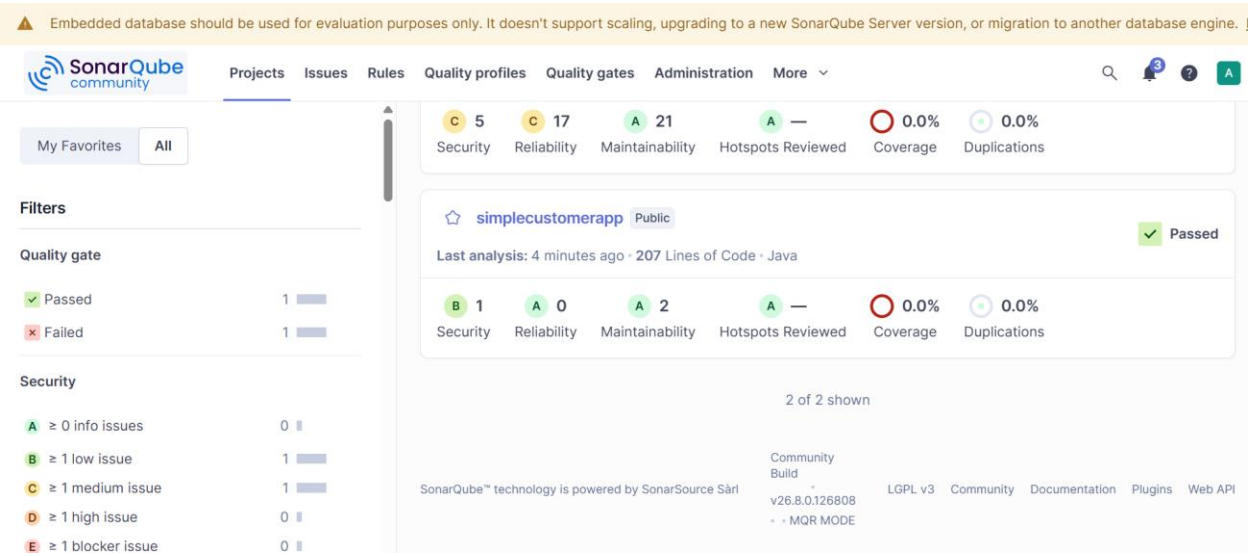
Step 3: Run the Job and Review Results

Click Build Now and open Console Output to follow each stage() block in order; the try/finally block guarantees the Slack Notification stage runs whether the build passes or fails.

Screenshots: Pipeline (simplecustomerapp-scripted).

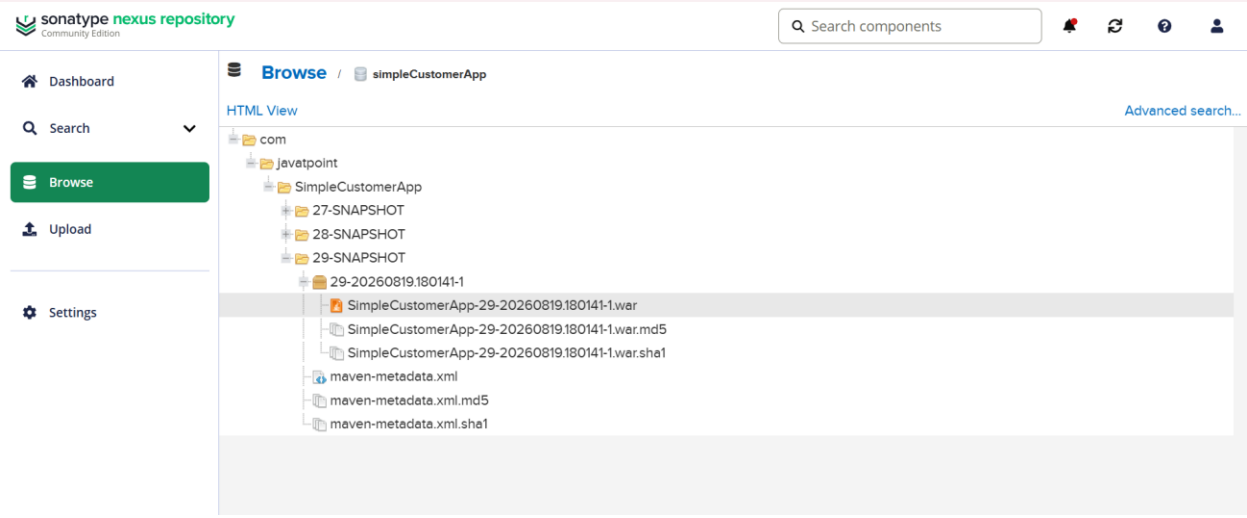
```
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/feature-1.1^{commit} # timeout=10
Checking out Revision 05222fef4b498f40dd73a8a1b6fbdb0c619d398a (refs/remotes/origin/feature-1.1)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 05222fef4b498f40dd73a8a1b6fbdb0c619d398a # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git branch -D feature-1.1 # timeout=10
 > git checkout -b feature-1.1 05222fef4b498f40dd73a8a1b6fbdb0c619d398a # timeout=10
Commit message: "Update pom.xml"
 > git rev-list --no-walk 05222fef4b498f40dd73a8a1b6fbdb0c619d398a # timeout=10
[Pipeline] echo
Git clone completed successfully
```

```
84ea257cf130
18:01:35.027 INFO  Analysis total time: 4.460 s
18:01:35.028 INFO  SonarScanner Engine completed successfully
18:01:35.056 INFO  EXECUTION SUCCESS
18:01:35.057 INFO  Total time: 11.476s
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
SonarQube analysis completed
```

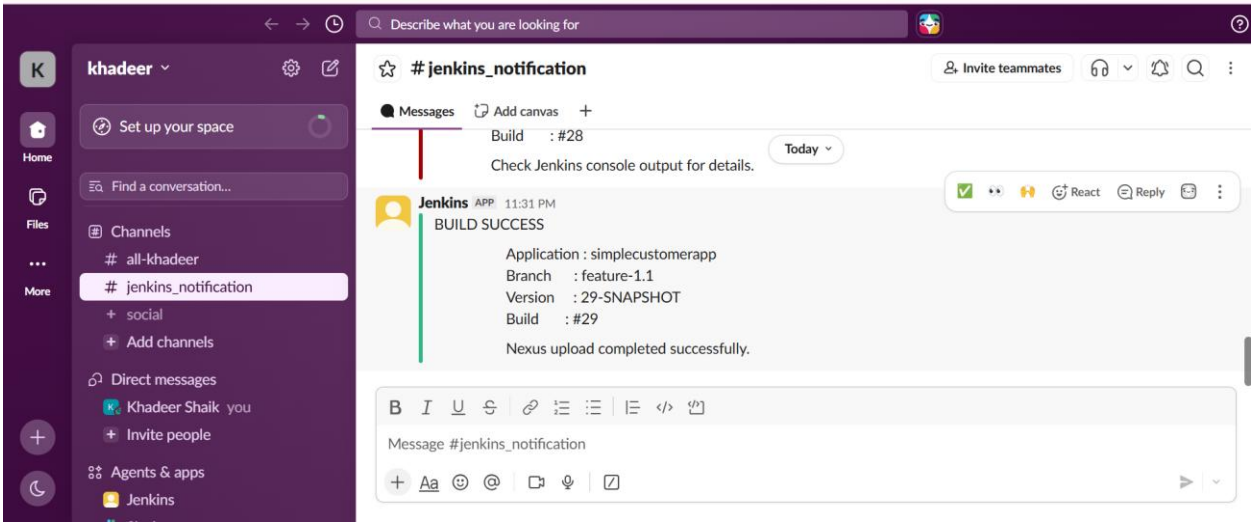


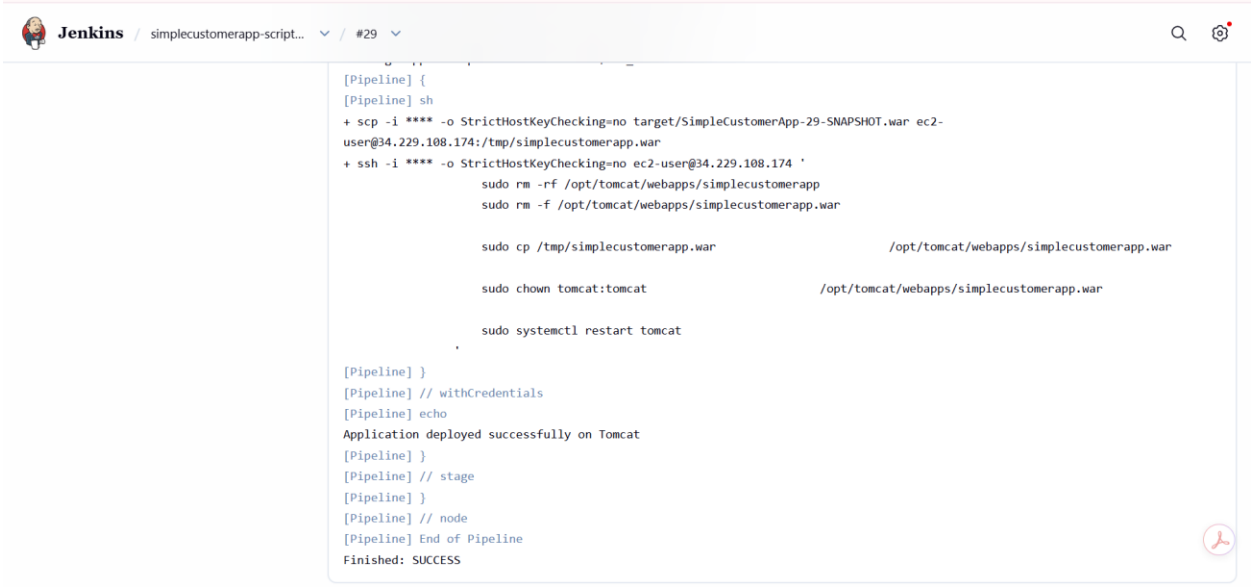
```
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ SimpleCustomerApp ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- war:3.3.2:war (default-war) @ SimpleCustomerApp ---
[INFO] Packaging webapp
[INFO] Assembling webapp [SimpleCustomerApp] in [/var/lib/jenkins/workspace/111/target/SimpleCustomerApp-29-SNAPSHOT]
[INFO] Processing war project
[INFO] Building war: /var/lib/jenkins/workspace/111/target/SimpleCustomerApp-29-SNAPSHOT.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.326 s
[INFO] Finished at: 2026-08-19T18:01:38Z
[INFO] ------------------------------------------------------------------------
```

```
Version: 29-SNAPSHOT
File: SimpleCustomerApp-29-SNAPSHOT.war
Repository:simpleCustomerApp
Downloading: http://34.229.108.174:8081/repository/simpleCustomerApp/com/javatpoint/SimpleCustomerApp/29-SNAPSHOT/maven-metadata.xml
Uploading: http://34.229.108.174:8081/repository/simpleCustomerApp/com/javatpoint/SimpleCustomerApp/29-SNAPSHOT/SimpleCustomerApp-29-20260819.180141-1.war
100 % completed (1.4 kB / 1.4 kB).
Uploaded: http://34.229.108.174:8081/repository/simpleCustomerApp/com/javatpoint/SimpleCustomerApp/29-SNAPSHOT/SimpleCustomerApp-29-20260819.180141-1.war (1.4 kB at 36 kB/s)
Uploading: http://34.229.108.174:8081/repository/simpleCustomerApp/com/javatpoint/SimpleCustomerApp/29-SNAPSHOT/maven-metadata.xml
Uploaded: http://34.229.108.174:8081/repository/simpleCustomerApp/com/javatpoint/SimpleCustomerApp/29-SNAPSHOT/maven-metadata.xml (602 B at 18 kB/s)
Uploading artifact SimpleCustomerApp-29-SNAPSHOT.war completed.
[Pipeline] echo
Artifact uploaded to Nexus successfully
```




```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: khadeer, channel: C0BQWR8R0QG, color: good,
botUser: true, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
```



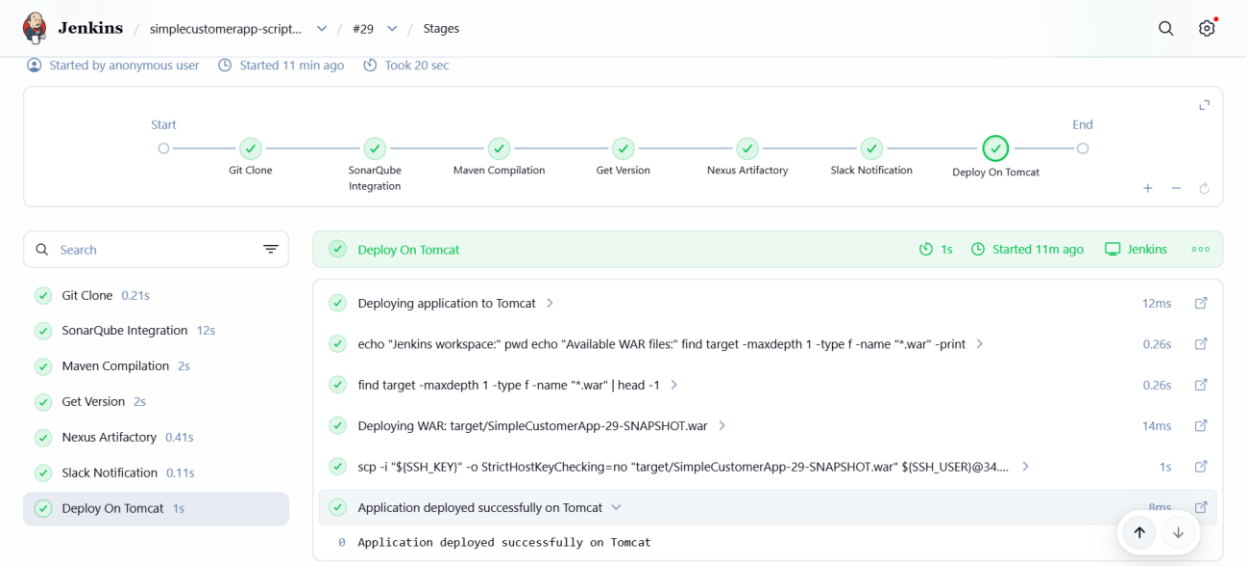


Manager

List Applications	HTML Manager Help	Manager Help	Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/simplecustomerapp	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes



Validation Steps

- Console Output shows all five stage() blocks executing, followed by the Slack Notification stage even on a forced failure.
- The checked-out workspace matches the feature-1.1 branch.
- SonarQube, Nexus, and Tomcat reflect the new build exactly as in Task 2.
- Slack posts a FAILURE-colored message if a stage is intentionally broken to test the finally block, and a SUCCESS-colored message on a clean run.

Explanation

Scripted pipelines run as arbitrary Groovy inside a single node {} block, so stage() calls are just labeled sections of imperative code rather than a fixed schema. This gives direct access to try/catch/finally, letting the Slack Notification logic live in a finally block that always runs, which is more verbose than Declarative's built-in post {} section but offers finer control over error handling and branching.

Real-Time Use Case

Scripted syntax is commonly kept for pipelines that need conditional stage skipping, dynamic stage generation (looping over a list of modules), or custom retry/backoff logic that does not map cleanly onto Declarative's fixed directives, while still notifying Slack on every outcome via a shared finally block.

Common Errors and Troubleshooting

Error	Cause	Resolution
groovy.lang.MissingMethodException on git	Missing branch or url map keys	Pass both branch and url exactly as shown to the git step
Slack step never runs after failure	slackSend placed inside try instead of finally	Move the Slack Notification stage into the finally block
tool 'maven-3.9' not found	Tool name does not match Manage Jenkins -> Tools	Match the tool name string exactly, case-sensitive
Script not yet approved	Sandbox blocks a method used in the script	Approve the signature under Manage Jenkins -> In-process Script Approval, or simplify the Groovy

Task 4: Sample Skeleton of Pipelines

Outcome

Two reusable, minimal skeletons - one Declarative, one Scripted - that give the blank starting structure for any future pipeline job, with the six standard stage names already stubbed in as comments/placeholders.

Objective

To provide a quick-reference template so new Jenkinsfiles are started from a consistent skeleton instead of copy-pasting a full working pipeline each time.

Declarative Pipeline Skeleton

```
pipeline {
    agent any                 // or: agent { docker { image '...' } }


    tools {
        // maven 'maven-3.9'
    }

    stages {
        stage('Git Clone') {
            steps { /* git branch:, url: */ }
        }
        stage('SonarQube Integration') {
            steps { /* withSonarQubeEnv(...) { sh '...' } */ }
        }
        stage('Maven Compilation') {
            steps { /* sh 'mvn clean package' */ }
        }
        stage('Nexus Artifactory') {
            steps { /* mvn deploy:deploy-file ... */ }
        }
        stage('Slack Notification') {
            steps { /* slackSend(...) */ }
        }
        stage('Deploy On Tomcat') {
            steps { /* curl -T target/app.war ... */ }
        }
    }

    post {
        success { /* slackSend(color: 'good', ...) */ }
        failure { /* slackSend(color: 'danger', ...) */ }
        always  { /* cleanWs() */ }
    }
}
```

Scripted Pipeline Skeleton

```
node {
    try {
        stage('Git Clone') {
            // git branch: '...', url: '...'
        }
        stage('SonarQube Integration') {
            // withSonarQubeEnv('sonar-server') { sh '...' }
        }
        stage('Maven Compilation') {
            // sh 'mvn clean package'
```

```
        }
        stage('Nexus Artifactory') {
            // sh 'mvn deploy:deploy-file ...'
        }
        stage('Deploy On Tomcat') {
            // sh 'curl -T target/app.war ...'
        }
    } catch (err) {
        currentBuild.result = 'FAILURE'
        throw err
    } finally {
        stage('Slack Notification') {
            // slackSend(...)
        }
    }
}
```

Explanation

Both skeletons keep the same six stage names used in Tasks 1-3 so a new project can be scaffolded by filling in the commented placeholders rather than redesigning the stage list. The Declarative skeleton is preferred for straightforward, linear pipelines; the Scripted skeleton is preferred once conditional logic, loops, or custom error handling are needed.

Task 5: Parameterized Job (spring3-mvc-maven-xml-hello-world-1)

Outcome

A parameterized Jenkins Pipeline job named spring3-mvc-hello-world-param that prompts for a Git branch, a target deployment environment, and whether to run tests, then clones, builds, and optionally deploys spring3-mvc-maven-xml-hello-world-1 according to those choices.

Objective

To demonstrate Jenkins build parameters so the same job definition can build different branches and target different environments without editing the Jenkinsfile, satisfying a common request for developer-triggered, configurable builds.

Prerequisites

- Jenkins master with the Pipeline and Git plugins installed.
- A Maven installation registered under Manage Jenkins -> Tools, or a Docker Maven image as used in earlier tasks.
- Read access to 'https://github.com/salmansbab94/spring3-mvc-maven-xml-hello-world-1.git'.

Step-by-Step Implementation

Step 1: Create the Pipeline Job and Enable 'This project is parameterized'

Parameter	Value
Location	New Item -> Pipeline
Item name	spring3-mvc-hello-world-param
Option	This project is parameterized (checked)

Step 2: Define the Build Parameters

Parameter	Type	Default / Choices
BRANCH	String Parameter	master

RUN_TESTS	Boolean Parameter	true

Step 3: Add the Parameterized Jenkinsfile

```
node {

    // =========================================================
    // CONFIGURATION
    // =========================================================

    def APP_NAME = "spring3-mvc-maven-xml-hello-world"

    def SONAR_PROJECT_KEY = "spring3-mvc-maven-xml-hello-world"
    def SONAR_HOST = "http://50.17.59.76:9000"
    def SONAR_SCANNER = "/opt/sonar_scanner/bin/sonar-scanner"

    def NEXUS_URL = "34.229.108.174:8081"
    def NEXUS_REPOSITORY = "hello-world"

    def TOMCAT_HOST = "34.229.108.174"
    def TOMCAT_DEPLOY_PATH = "/opt/tomcat/webapps"

    try {

        // =====================================================
        // 1. GIT CLONE
        // =====================================================

stage('Git Clone') {

    // If BRANCH_NAME is not supplied, use main
    def BRANCH = params.BRANCH_NAME ?: 'master'

    echo "======================================="
    echo "Git Clone"
    echo "======================================="
    echo "Parameter BRANCH_NAME : ${params.BRANCH_NAME}"
    echo "Selected branch       : ${BRANCH}"

    git(
        branch: BRANCH,
        url: 'https://github.com/salmansbab94/spring3-mvc-maven-xml-hello-world-1.git'
    )

    echo "Git clone completed successfully"
}



        // =====================================================
        // 2. SONARQUBE INTEGRATION
        // =====================================================

        stage('SonarQube Integration') {

            if (params.SONAR_ANALYSIS) {
```

```
        echo "Starting SonarQube analysis"

        withCredentials([
            string(
                credentialsId: 'sonar-token',
                variable: 'SONAR_TOKEN'
            )
        ]) {

            sh """
                ${SONAR_SCANNER} \
                -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
                -Dsonar.projectName=${APP_NAME} \
                -Dsonar.sources=src \
                -Dsonar.host.url=${SONAR_HOST} \
                -Dsonar.token=\${SONAR_TOKEN}
            """
        }

        echo "SonarQube analysis completed"

    } else {

        echo "SonarQube analysis skipped"

    }
}


// ======================================================
// 3. MAVEN COMPILATION
// ======================================================

stage('Maven Compilation') {

    def mvnHome = tool 'Maven'

    echo "Using Maven: ${mvnHome}"

    sh """
        ${mvnHome}/bin/mvn clean package -DskipTests
    """

    echo "Maven build completed successfully"
}


// ======================================================
// 4. GET MAVEN VERSION
// ======================================================

stage('Get Version') {

    def mvnHome = tool 'Maven'
```

```
    env.APP_VERSION = sh(
        script: """
            ${mvnHome}/bin/mvn \
            help:evaluate \
            -Dexpression=project.version \
            -q \
            -DforceStdout
        """,
        returnStdout: true
    ).trim()

    echo "Application Version: ${env.APP_VERSION}"
}


// =====================================================
// 5. FIND WAR FILE
// =====================================================

stage('Find WAR') {

    sh '''
        echo "Target directory:"
        ls -lh target/

        echo "WAR files:"
        find target -maxdepth 1 -type f -name "*.war"
    '''

    env.WAR_FILE = sh(
        script: '''
            find target -maxdepth 1 -type f -name "*.war" | head -1
        ''',
        returnStdout: true
    ).trim()

    if (!env.WAR_FILE) {
        error "No WAR file found in target directory"
    }

    env.WAR_NAME = sh(
        script: "basename '${env.WAR_FILE}'",
        returnStdout: true
    ).trim()

    echo "WAR file found: ${env.WAR_NAME}"
}


// =====================================================
// 6. NEXUS ARTIFACTORY
// =====================================================

stage('Nexus Artifactory') {

    echo "Uploading WAR to Nexus"
```

```
            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: NEXUS_URL,
                groupId: 'com.javatpoint',
                version: env.APP_VERSION,
                repository: NEXUS_REPOSITORY,
                credentialsId: 'nexus-credentials',
                artifacts: [[
                    artifactId: APP_NAME,
                    classifier: '',
                    file: env.WAR_FILE,
                    type: 'war'
                ]]
            )

            echo "WAR uploaded to Nexus successfully"
        }


        // =====================================================
        // 7. SLACK NOTIFICATION
        // =====================================================

        stage('Slack Notification') {

            slackSend(
                channel: 'jenkins_notification',
                color: 'good',
                message: """
BUILD SUCCESS

Application : ${APP_NAME}
Branch      : ${params.BRANCH_NAME}
Version     : ${env.APP_VERSION}
Build       : #${env.BUILD_NUMBER}
WAR         : ${env.WAR_NAME}

Nexus upload completed successfully.
"""
            )
        }


        // =====================================================
        // 8. DEPLOY ON TOMCAT
        // =====================================================

        stage('Deploy On Tomcat') {

            if (params.DEPLOY_TO_TOMCAT) {

                echo "Deploying ${env.WAR_NAME} to Tomcat"

                withCredentials([
```

```
                sshUserPrivateKey(
                    credentialsId: 'nexus-tomcat-ssh',
                    keyFileVariable: 'SSH_KEY',
                    usernameVariable: 'SSH_USER'
                )
            ]) {

                sh """
                    scp -i "\${SSH_KEY}" \
                        -o StrictHostKeyChecking=no \
                        "${env.WAR_FILE}" \
                        \${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

                    ssh -i "\${SSH_KEY}" \
                        -o StrictHostKeyChecking=no \
                        \${SSH_USER}@${TOMCAT_HOST} '

                        sudo rm -rf \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}

                        sudo rm -f \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo cp \
                            /tmp/${APP_NAME}.war \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo chown tomcat:tomcat \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo systemctl restart tomcat

                        sudo systemctl status tomcat --no-pager

                    '
                """
            }

            echo "Tomcat deployment completed successfully"

        } else {

            echo "Tomcat deployment skipped"

        }
    }


    // =====================================================
    // SUCCESS
    // =====================================================

    echo """
========================================
PIPELINE COMPLETED SUCCESSFULLY
========================================
```

```
Application : ${APP_NAME}
Branch      : ${params.BRANCH_NAME}
Version     : ${env.APP_VERSION}
WAR         : ${env.WAR_NAME}
Build       : #${env.BUILD_NUMBER}

========================================
"""


    } catch (err) {

        // ====================================================
        // FAILURE NOTIFICATION
        // ====================================================

        echo "Pipeline failed: ${err}"

        slackSend(
            channel: 'C0BQWR8R0Q',
            color: 'danger',
            message: """
BUILD FAILED

Application : ${APP_NAME}
Branch      : ${params.BRANCH_NAME}
Build       : #${env.BUILD_NUMBER}

Check Jenkins console output for details.
"""
        )

        throw err
    }
}
```

Step 4: Run With Parameters

Click Build with Parameters, choose the BRANCH, DEPLOY_ENV, and RUN_TESTS values for this run, then Build; Jenkins injects each choice into the run as params.<NAME>.

Screenshot: 'This project is parameterized' job .

This project is parameterized

String Parameter

Name

BRANCH_NAME

Default Value

master

Description

https://github.com/salmansbab94/spring3-mvc-maven-xml-hello-world-1.git

Boolean Parameter

Name

SONAR_ANALYSIS

Set by Default

Description

Boolean Parameter

Name

DEPLOY_TO_TOMCAT

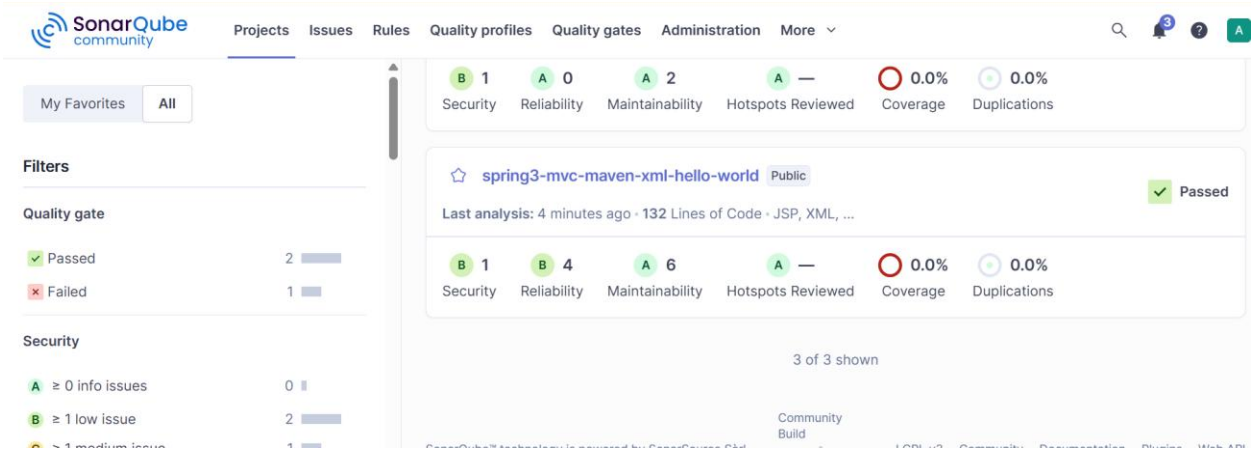
Set by Default

Description

Plain text Preview

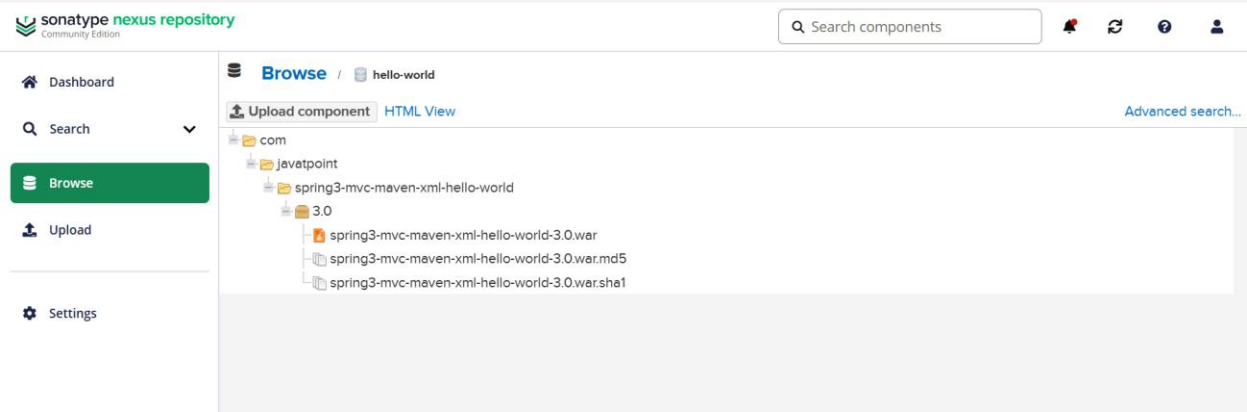
```
Fetching upstream changes from https://github.com/salmansbab94/spring3-mvc-maven-xml-hello-world-1.git
 > git --version # timeout=10
 > git --version # 'git version 2.50.1'
 > git fetch --tags --force --progress -- https://github.com/salmansbab94/spring3-mvc-maven-xml-hello-world-1.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision b60ee9362809a28991bbaa3b5791c920d44ae5f1 (refs/remotes/origin/master)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f b60ee9362809a28991bbaa3b5791c920d44ae5f1 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git branch -D master # timeout=10
 > git checkout -b master b60ee9362809a28991bbaa3b5791c920d44ae5f1 # timeout=10
Commit message: "Update pom.xml for Maven project configuration"
 > git rev-list --no-walk b60ee9362809a28991bbaa3b5791c920d44ae5f1 # timeout=10
[Pipeline] echo
Git clone completed successfully
```

```
19:35:06.937 INFO  More about the report processing at http://50.17.59.76:9000/api/ce/task?id=11ad4866-54a8-475d-9c2d-cc51f77f396b
19:35:07.042 INFO  Analysis total time: 9.991 s
19:35:07.044 INFO  SonarScanner Engine completed successfully
19:35:07.072 INFO  EXECUTION SUCCESS
19:35:07.073 INFO  Total time: 17.284s
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
SonarQube analysis completed
```

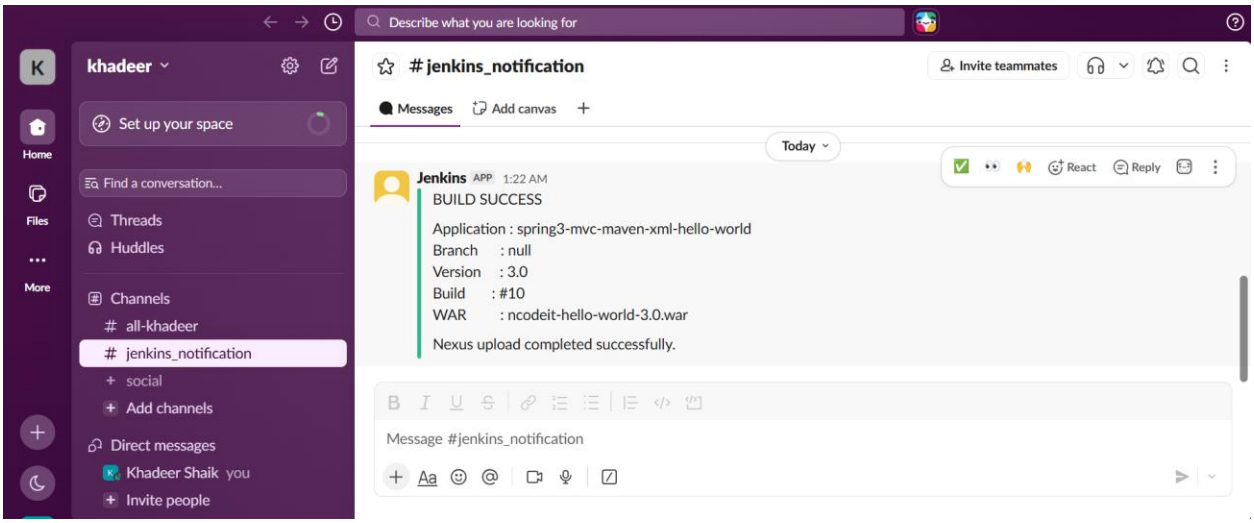


```
[INFO] --- surefire:3.5.4:test (default-test) @ ncodeit-hello-world ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- war:3.4.0:war (default-war) @ ncodeit-hello-world ---
[INFO] Packaging webapp
[INFO] Assembling webapp [ncodeit-hello-world] in [/var/lib/jenkins/workspace/parameterized job/target/ncodeit-hello-world-3.0]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/parameterized job/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/parameterized job/target/ncodeit-hello-world-3.0.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.184 s
[INFO] Finished at: 2026-08-19T19:35:11Z
[INFO] ------------------------------------------------------------------------
[Pipeline] echo
Maven build completed successfully
```

```
Uploading: http://34.229.108.174:8081/repository/hello-world/com/javatpoint/spring3-mvc-maven-xml-hello-world/3.0/spring3-mvc-maven-xml-hello-world-3.0.war
10 % completed (451 kB / 4.2 MB).
20 % completed (860 kB / 4.2 MB).
30 % completed (1.3 MB / 4.2 MB).
40 % completed (1.7 MB / 4.2 MB).
50 % completed (2.1 MB / 4.2 MB).
60 % completed (2.5 MB / 4.2 MB).
70 % completed (2.9 MB / 4.2 MB).
80 % completed (3.4 MB / 4.2 MB).
90 % completed (3.8 MB / 4.2 MB).
100 % completed (4.2 MB / 4.2 MB).
Uploaded: http://34.229.108.174:8081/repository/hello-world/com/javatpoint/spring3-mvc-maven-xml-hello-world/3.0/spring3-mvc-maven-xml-hello-world-3.0.war (4.2 MB at 34 MB/s)
Uploading artifact ncodeit-hello-world-3.0.war completed.
[Pipeline] echo
WAR uploaded to Nexus successfully
```




```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: khadeer, channel: jenkins_notification, color: good, botUser: true, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
```




```
Djava.io.tmpdir=/opt/tomcat/temp org.apache.catalina.startup.Bootstrap start

Aug 19 19:35:16 ip-172-31-46-214.ec2.internal systemd[1]: Starting tomcat.service - Apache Tomcat 9...
Aug 19 19:35:16 ip-172-31-46-214.ec2.internal startup.sh[32553]: Tomcat started.
Aug 19 19:35:16 ip-172-31-46-214.ec2.internal systemd[1]: Started tomcat.service - Apache Tomcat 9.
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
Tomcat deployment completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo

========================================
PIPELINE COMPLETED SUCCESSFULLY
========================================

Application : spring3-mvc-maven-xml-hello-world
Branch      : null
Version     : 3.0
WAR         : ncodeit-hello-world-3.0.war
Build       : #9

========================================
```

Not secure 34.229.108.174:8080/manager/html

Message:	OK

Manager

List Applications	HTML Manager Help	Manager Help	Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	*None specified*	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	*None specified*	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	*None specified*	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	*None specified*	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	*None specified*	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/simplecustomerapp	*None specified*		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/spring3-mvc-maven-xml-hello-world	*None specified*	Spring3 MVC Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Not secure 34.229.108.174:8080/spring3-mvc-maven-xml-hello-world/

Spring 3 MVC Project

Welcome Welcome!

Learn more

Heading
ABC
View details

Heading
ABC
View details

Heading
ABC
View details

© Mkyong.com 2015

Jenkins / spring3-mvc-hello-world-... / #9 / Stages

#9

Rerun

Started by anonymous user | Started 13 min ago | Took 27 sec | Parameters

Start → Git Clone → SonarQube Integration → Maven Compilation → Get Version → Find WAR → Nexus Artifactory → Slack Notification → Deploy On Tomcat → End

Search

Git Clone 0.68s
SonarQube Integration 18s
Maven Compilation 3s
Get Version 2s
Find WAR 0.84s
Nexus Artifactory 0.17s
Slack Notification 74ms

Deploy On Tomcat — 1s — Started 13m ago — Jenkins

Deploying ncodeit-hello-world-3.0.war to Tomcat — 15ms
Shell Script — 1s
Tomcat deployment completed successfully — 8ms

```
0  Tomcat deployment completed successfully
```

Validation Steps

- Build with Parameters prompts for BRANCH, DEPLOY_ENV, and RUN_TESTS before starting.
- Console Output confirms the branch actually checked out matches the BRANCH value entered.
- The Test stage is skipped in Console Output when RUN_TESTS is set to false.
- The Deploy stage echoes the DEPLOY_ENV value chosen for that run.

Explanation

The parameters {} block defines typed inputs (string, choice, booleanParam) that Jenkins renders as a form before the build starts; each value is exposed inside the pipeline as params.<NAME> and can drive branch selection, conditional stages via when { expression { ... } }, and environment-specific logic in a single job definition.

Real-Time Use Case

A shared 'build any branch, deploy anywhere' job like this lets developers self-serve a build of their own feature branch against a chosen environment without a Jenkins admin creating a new job per branch or per environment.

Common Errors and Troubleshooting

Error	Cause	Resolution
params.BRANCH is null	Job run directly instead of via Build with Parameters on first run	First run of a new parameterized job may need to run once to register parameters; re-run with parameters afterward
Test stage always runs	when { expression { ... } } evaluated before params bound, or wrong param name	Confirm the parameter name matches params.RUN_TESTS exactly, case-sensitive
Branch checkout fails	BRANCH value does not exist in the repo	Validate the branch name entered at build time against the remote

Task 6: Setting Up One Jenkins Slave (Agent) Machine

Outcome

A second Linux machine (build-agent-01) is attached to the Jenkins master as a permanent agent over SSH, labeled build-agent, so jobs can be pinned to it with agent { label 'build-agent' } instead of always running on the master.

Objective

To offload build execution from the Jenkins master onto a dedicated worker node, improving isolation and letting heavier or environment-specific builds (for example, ones needing Docker or a particular JDK) run on hardware suited to that purpose.

Prerequisites

- A second Linux host reachable from the Jenkins master over SSH, with Java (JDK 17) installed.
- A dedicated non-root user on the agent host (for example jenkins-agent) with a home directory to use as the remote root.
- An SSH keypair: the public key authorized on the agent host, the private key stored in Jenkins as an SSH Username with private key credential.
- The SSH Build Agents plugin installed on the master (bundled with Jenkins by default).

Step-by-Step Implementation

Step 1: Prepare the Agent Host

```
sudo dnf update
sudo dnf install java-21-amazon-corretto -y
sudo dnf install git -y
ssh-keygen
cd~/.ssh
cat id_rsa.pub > authorized_keys
chmod 700 authorized_keys
mkdir -p /var/lib/jenkins/.ssh
cd /var/lib/jenkins/.ssh
ssh-keyscan -H 3.81.15.93  >>/var/lib/jenkins/.ssh/known_hosts
chown jenkins:jenkins known_hosts
chmod 700 known_hosts
```

Step 2: Add the SSH Credential in Jenkins

Parameter	Value
Location	Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials
Kind	SSH Username with private key
ID	jenkins-agent-ssh
Username	jenkins-agent
Private Key	Enter directly (paste the matching private key)

Step 3: Create the New Node

Parameter	Value
Location	Manage Jenkins -> Nodes -> New Node
Node name	build-agent-01
Type	Permanent Agent
Remote root directory	/home/ec2-user/jenkins-agent/agent
Labels	java
Launch method	Launch agents via SSH
Host	<agent-host-ip>
Credentials	jagent01
Host Key Verification Strategy	Non verifying (or Known hosts file, for stricter setups)

📷 *Screenshots: Agent host prep (Java install, user, SSH key).*

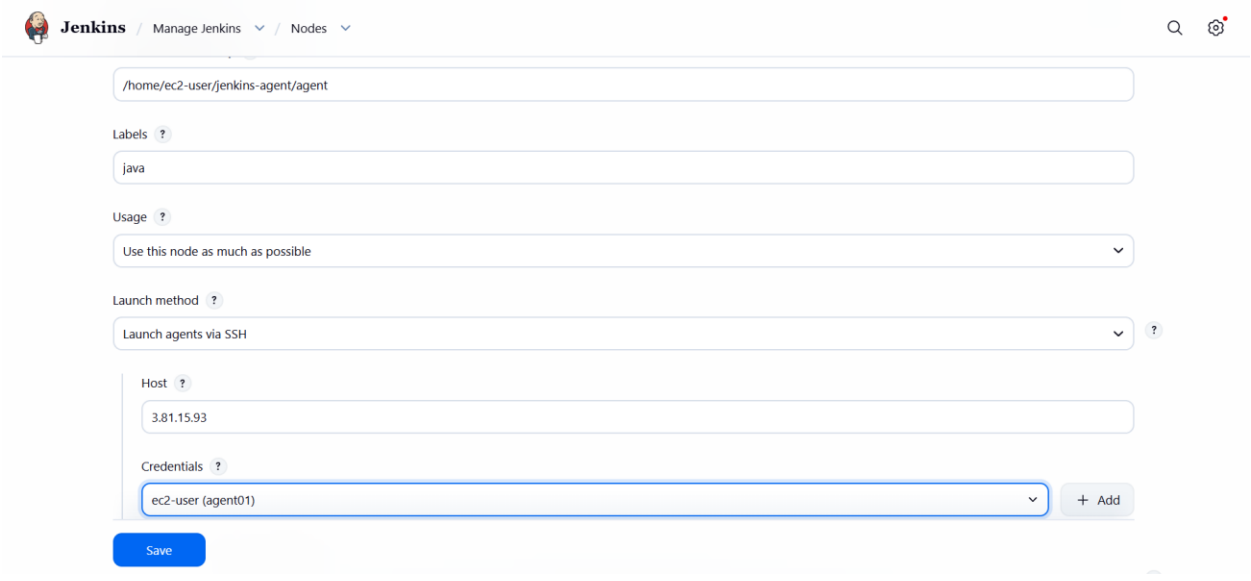
```
[ec2-user@ip-172-31-38-50 ~]$ sudo dnf install java-21-amazon-corretto -y
Amazon Linux 2023 Kernel Livepatch repository                    631 kB/s |  69 kB     00:00
Dependencies resolved.
================================================================================
 Package                    Architecture  Version                    Repository  Size
================================================================================
Installing:
 java-21-amazon-corretto    x86_64        1:21.0.12+8-1.amzn2023.1   amazonlinux 218 k
Installing dependencies:
 alsa-lib                   x86_64        1.2.7.2-1.amzn2023.0.3     amazonlinux 503 k
 cairo                      x86_64        1.18.0-4.amzn2023.0.3      amazonlinux 717 k
 dejavu-sans-fonts          noarch        2.37-16.amzn2023.0.2       amazonlinux 1.3 M
 dejavu-sans-mono-fonts     noarch        2.37-16.amzn2023.0.2       amazonlinux 467 k
 dejavu-serif-fonts         noarch        2.37-16.amzn2023.0.2       amazonlinux 1.0 M
 fontconfig                 x86_64        2.13.94-2.amzn2023.0.2     amazonlinux 273 k
 fonts-filesystem           noarch        1:2.0.5-12.amzn2023.0.2    amazonlinux 9.5 k
 freetype                   x86_64        2.13.2-5.amzn2023.0.2      amazonlinux 422 k
 giflib                     x86_64        5.2.1-9.amzn2023.0.4       amazonlinux 49 k
 google-noto-fonts-common   noarch        20240401-1.amzn2023.0.2    amazonlinux 17 k
 google-noto-sans-vf-fonts  noarch        20240401-1.amzn2023.0.2    amazonlinux 593 k
 graphite2                  x86_64        1.3.14-7.amzn2023.0.3      amazonlinux 97 k
 harfbuzz                   x86_64        7.0.0-2.amzn2023.0.2       amazonlinux 873 k
 java-21-amazon-corretto-headless x86_64  1:21.0.12+8-1.amzn2023.1   amazonlinux 96 M
 javapackages-filesystem    noarch        6.0.0-7.amzn2023.0.6       amazonlinux 12 k
 langpacks-core-font-en     noarch        3.0-21.amzn2023.0.4        amazonlinux 10 k
```

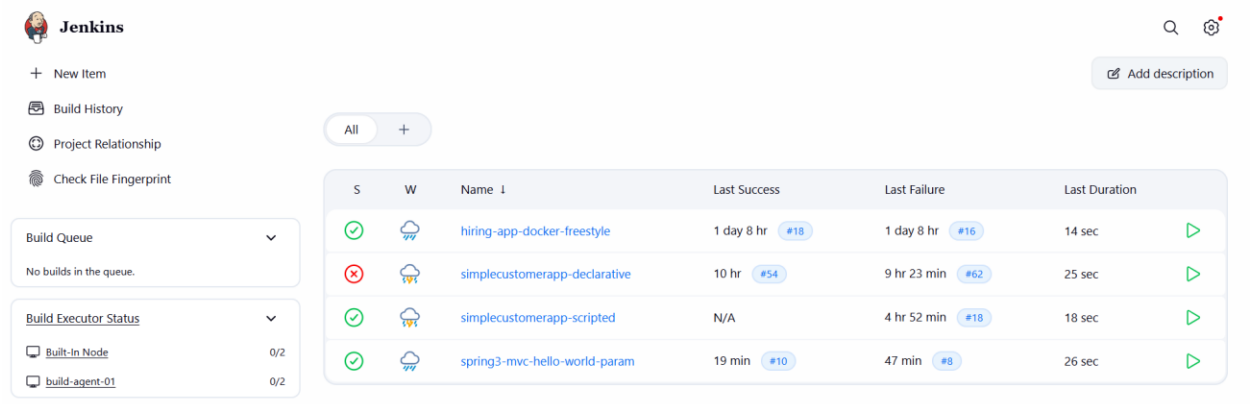
```
[ec2-user@ip-172-31-38-50 ~]$ sudo su
[root@ip-172-31-38-50 ec2-user]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:kfeZOE/mcPlqAHYNI0ZPPyU71n9dNZ3+BpzHBdfwa5A root@ip-172-31-38-50.ec2.internal
The key's randomart image is:
+---[RSA 3072]----+
|       .. . . ++*|
|        o+o. =.+*|
|       .o.o+*Eoo+|
|        oo.+.*=o*|
|       .So+ O  ==|
|          .O .. +|
|           .o .. |
|            ..   |
|           ..    |
+----[SHA256]-----+
```

```
[root@ip-172-31-38-50 ec2-user]# cd ~/.ssh
[root@ip-172-31-38-50 .ssh]# ll
total 12
-rw-------. 1 root root  555 Aug 19 19:56 authorized_keys
-rw-------. 1 root root 2622 Aug 19 20:01 id_rsa
-rw-r--r--. 1 root root  587 Aug 19 20:01 id_rsa.pub
[root@ip-172-31-38-50 .ssh]# cat id_rsa.pub > authorized_keys
[root@ip-172-31-38-50 .ssh]# chmod 700 authorized_keys
[root@ip-172-31-38-50 .ssh]# ll
total 12
-rwx------. 1 root root  587 Aug 19 20:02 authorized_keys
-rw-------. 1 root root 2622 Aug 19 20:01 id_rsa
-rw-r--r--. 1 root root  587 Aug 19 20:01 id_rsa.pub
[root@ip-172-31-38-50 .ssh]#
```

```
[ec2-user@ip-172-31-40-113 ~]$ sudo su
[root@ip-172-31-40-113 ec2-user]# ssh-keyscan -H 3.81.15.93 >>/var/lib/jenkins/.ssh/known_hosts
[root@ip-172-31-40-113 ec2-user]# chown jenkins:jenkins known_hosts
chown: cannot access 'known_hosts': No such file or directory
[root@ip-172-31-40-113 ec2-user]# cd /var/lib/jenkins/.ssh/
[root@ip-172-31-40-113 .ssh]# ll
total 12
-rw-------. 1 jenkins jenkins   93 Aug 19 14:39 authorized_keys
-rw-------. 1 jenkins jenkins 3541 Aug 19 20:04 known_hosts
-rw-r--r--. 1 jenkins jenkins   95 Aug 18 15:40 known_hosts.old
[root@ip-172-31-40-113 .ssh]# chown jenkins:jenkins known_hosts
[root@ip-172-31-40-113 .ssh]# ll
total 12
-rw-------. 1 jenkins jenkins   93 Aug 19 14:39 authorized_keys
-rw-------. 1 jenkins jenkins 3541 Aug 19 20:04 known_hosts
-rw-r--r--. 1 jenkins jenkins   95 Aug 18 15:40 known_hosts.old
[root@ip-172-31-40-113 .ssh]# chmod 700 known_hosts
[root@ip-172-31-40-113 .ssh]# ll
total 12
-rw-------. 1 jenkins jenkins   93 Aug 19 14:39 authorized_keys
-rwx------. 1 jenkins jenkins 3541 Aug 19 20:04 known_hosts
-rw-r--r--. 1 jenkins jenkins   95 Aug 18 15:40 known_hosts.old
[root@ip-172-31-40-113 .ssh]#
```

Screenshots: New Node configuration screen.





Step 4: Verify Connectivity and Pin a Job to the Agent

Save the node and open its status page; Jenkins connects over SSH and shows the agent as online with its Java and OS details logged. Pin a job to it with:

```
pipeline {
    agent { label 'java' }
    stages {
        stage('Verify') {
            steps { sh 'hostname && java -version' }
        }
    }
}
```

Validation Steps

- Manage Jenkins -> Nodes shows agent-01 with a green (online) icon.
- The node's log shows a successful SSH connection and remoting.jar launch with no handshake errors.
- A test job with agent { label 'java' } runs on build-agent-01, confirmed by the hostname printed in Console Output.

Explanation

Jenkins launches an agent over SSH by copying its remoting.jar to the remote root directory and starting it as a background process under the configured user; once online, the master schedules any job whose agent {} or node {} block requests a matching label onto that machine instead of running it locally.

Real-Time Use Case

Teams add dedicated build agents to isolate untrusted or resource-heavy builds from the Jenkins master, to provide a specific OS/toolchain (for example, a Docker-capable Linux box) that the master itself does not have, or to scale build throughput horizontally as job volume grows.

Common Errors and Troubleshooting

Error	Cause	Resolution
Node shows offline: Authentication failed	Public key not authorized, or wrong username in the credential	Confirm the public key is in authorized_keys for the exact username configured

No Java executable found	JDK not installed, or not on PATH for the agent user	Install JDK 21 on the agent host and confirm java -version works as that user
Node connects then drops immediately	Remote root directory not writable by the agent user	chown the remote root directory to the SSH user before retrying
Job stays queued, never runs on the agent	Label mismatch between the job's agent{} block and the node's Labels field	Match the label string exactly, case-sensitive